

Pre Course Material

Health Data Analytics and Machine Learning

2026-08-26

Contents

Welcome to HDA	5
What Makes HDA Unique	5
Program Overview	6
Getting Ready For the Program	6
Iterative Project Development	7
1 Term Dates and Time Requirements	9
1.1 Imperial College Term Dates	9
2 General Advice	11
2.1 Advice from the Teaching Team	12
2.2 Advice from previous HDA students	13
3 Getting to Know Your Computer	15
3.1 Computer Specifications	15
3.2 The Black Box	18
4 Setting Up Your Computer	21
4.1 Bash	21
4.2 R	22
4.3 Python	26
4.4 SQL	30
4.5 Git	30
4.6 Other Programming Languages	31
5 Prep Material	33
5.1 Molecular Biology	33
5.2 Quantitative Skills	34
5.3 Programming	35
5.4 Fun Readings	36
6 Course Preview	39
Term 1	39
Term 1 and Term 2	42

Term 2	43
Term 3	46
7 Reading List	49
7.1 Statistics	49
7.2 Programming	50
7.3 Epidemiology	50
8 FAQ	51
1. Do you have any suggestions for finding accommodation in London?	51
2. How much money will I need to live and study in London for a year?	51
3. What is my tuition fee status?	52
4. How and when should I send official documents (e.g. degree certificates)?	52
5. When does the course start?	52
6. I'm an international student – what else should I consider or do before my arrival?	52
7. Where does teaching take place?	53
8. Do you have a reading list of books to prepare for the course? . . .	53
9. Is there other preparation that I should do before starting the course?	53
10. Will I need to buy a calculator?	53
11. What books do I need to buy for the course?	54
12. Do I need to choose my own summer project topic?	54
9 Course Contact Information	55
10 Mandatory Training	57
10.1 Data Protection Awareness	57
10.2 Plagiarism Awareness	57
10.3 Attributes and Aspirations (Recommended)	58
11 Installing R and R Studio	59
11.1 Installing R	59
11.2 Installing RStudio	61
11.3 Installing Packages	61
12 Installing Python, Anaconda, and VSCode	67
12.1 Python	68
12.2 Python IDEs	68
12.3 Package Installation	72
13 Installing MySQL and DBeaver	75
13.1 MySQL Installation	75
13.2 DBeaver Installation	76
14 Installing Git, Github, and Github Desktop	83
14.1 Git	83

<i>CONTENTS</i>	5
14.2 Github	84
14.3 Github Desktop	85
15 Change Log	87
v.0.0	87
v.0.1	87
v.0.2	87

Welcome to HDA

Congratulations on your acceptance to the Health Data Analytics and Machine Learning (HDA) MSc at Imperial College London. Your impressive transcripts, CV, personal statement, and references stood out from well over 700 other applicants in a blind selection process. Each year we are excited to welcome a diverse cohort of exceptional candidates from all over the world. Our program is committed to making this formative year in your life as enjoyable, educational, and memorable as possible.

This pre-course guide will ensure that you are set up for success. It provides an introduction to the program, key term dates, general advice, pre-course instructions, and answers for some frequently asked questions.

What Makes HDA Unique

The students Each cohort is diverse and different. We accept candidates from a wide range of educational backgrounds such as biology, math, medicine, computer science, and more. Some have just graduated, some have substantial professional experience, and others already hold MSc or PHD degrees. Our students have been from a wide range of age, nationalities, and cultures. They typically have had vastly different life experiences. The only similarities between our students is that they are exceptional, open minded, and hard working. This diversity is a major strength of our program. The students benefit from each others perspective. The instructors get to engage with new ideas each year. Group research projects can achieve holistic and novel solutions.

The multidisciplinary approach Health data poses the biggest of big data challenges. Attempting to find causal signals of disease from a lifetime's worth of external exposures (pollution, stress, early life gender identity, etc.) or to identify molecular signatures of heterogeneous disease phenotypes requires more than just advanced analytical techniques. The challenge requires a rigorous and formal scientific approach which leverages transferable high performance computing and advanced statistical, machine learning, and AI methodology to find the signal through the noise. Possessing a core competency in a relevant domain such as molecular biology is necessary to ensure interpretable, actionable, and

impactful solutions.

The hands-on application with real data Real data is messy and real world problems do not have straight forward solutions. Attempting to solve problems without known solutions is often an unique experience for students. Through closely supervised projects in Term 2 and Term 3, you will get the experience of handling unexpected issues, formulating creative solutions, and understanding unexpected results while performing novel, cutting-edge research with real world clinical data.

These unique circumstances poses their own challenge: How to design a course that covers a wide range of advanced topics to a level that is rigorous enough for those familiar with the topic but accessible enough for those new to the material?

Program Overview

HDA is split into four components. The pre-course material defines the scope of the course and provides conceptual introductions to important core concepts. Term 1 up-skills the cohort and establishes the base competency needed for more advanced approaches. Optional short courses in Biology, Python, and Math are available. At this point, every student has the same multidisciplinary base skill set in coding, statistics, molecular biology, research, and data science. Term 2 builds on that base to learn and apply cutting-edge analytical methodology. Term 3 pairs each student with a world leading research team to design, implement, and report a novel solution to a real-world scientific issue. This capstone project offers an opportunity to demonstrate the skillset from the course and your new ability as an analytical expert.

Each course broadly follows the format of a 1.5-2 hour theoretical lecture in the morning followed by a 3 hour afternoon practical session to apply the subject of the lecture. There are optional weekly conceptual quizzes and ungraded midterms to help you figure out which subjects need more of your attention.

Getting Ready For the Program

Consistency is key. HDA requires a sustained effort throughout the entirety of the program. This requires prioritization and a manageable work load. You will be studying several courses at once, have a week of exams, and will be exploring your own research ideas. There will always be more work that you could do, another subject to review, or another line of code to write. Burning yourself out will lead to worse grades, learning outcomes, and enjoyment than a committed, organized, and achievable approach.

Don't panic. Don't feel that you need to be an expert in the subjects you will be taught before you arrive. A large number of the HDA cohorts have not been

provided pre-course material and have performed quite well. Student feedback was that a gentle conceptual introduction to the core themes of the course would have made new or advanced topics more easy to digest.

Please spend 30 minutes to an hour, 5-6 days a week, reviewing the suggested preparatory material. Note that we are not asking you to spend 300 hours, cover a certain amount of material, or vary your effort by when you receive this guide. Try to identify which areas you know the least and begin to get familiar with the subjects. Adopt a targeted approach on a small concept to make the most of each study session. For example: don't sit down to study linear algebra, but do sit down to study vectors (a concept within linear algebra). Establishing the good habit of consistent and manageable effort is just as important as the material you will cover.

Iterative Project Development

Most softwares, research projects, and programs adopt iterative project development. This is why there are multiple versions of software. This guide also has versions. Please see the change log for details on what has changed in each version update.

The current version of this guide is **v.0.2**

Chapter 1

Term Dates and Time Requirements

The Health Data Analytics and Machine Learning MSc is a **full-time, in-person, one-year course**. You **must** be in London and attend class during all of the HDA Term Dates.

During Term 1 and Term 2, classes will take place between 9:30am and 5pm on Monday, Tuesday, Thursday, and Friday (exact times vary depending on the module). Wednesdays will either have a lecture in the morning, optional short courses, or time for independent study. Term 3 is a full-time research project where you will still be required to be in London, but will be allowed a 2 week vacation (this is in addition to the time off in between terms). All exams and presentations are held in person.

1.1 Imperial College Term Dates

Below are provisional Imperial College terms dates for 2026-27. **Please note that some assessments submission and examination dates fall outside regular College term dates.**

HDA Term Dates

Event	Start Date	End Date
Induction Week	28 September 2026	2 October 2026
Term 1	5 October 2026	11 December 2026
Exam period 1	4 January 2027	8 January 2027
Term 2	11 January 2027	2 April 2027
Exam period 2 (Final Exams)	19 April 2027	23 April 2026

Event	Start Date	End Date
Exam period 2 (Project Due)	26 April 2027	30 April 2026
Term 3	3 May 2027	10 September 2027

Examination dates

Exam Period	Start Date	End Date
Exam period 1	4 January 2027	8 January 2027
Exam period 2 (Final Exams)	19 April 2027	23 April 2027
Exam period 2 (Project Due)	26 April 2027	30 April 2027
Final Presentation Date	6 September 2027	10 September 2027

Note: You must be in London for all assessments and please note these dates might be subject to change.

College Term dates

Imperial closure dates can be found [here](#).

Again, please note that as a one year postgraduate course, assessment submissions may take place outside the main term / exam periods.

Chapter 2

General Advice

There is no one best way to approach the course. This is a theme in the program. There is no template for research, education, or life. Each problem is unique and has multiple possible solutions.

Personalized medicine is a dominant theme in healthcare. Take asthma as an example. Historically, asthma was a single diagnosis but with multiple types of triggers and symptoms (a heterogeneous disease). Recently, a subtype has been identified: eosinophilic asthma. Those suffering from this subtype of asthma has previously been given a treatment that worked for the majority of asthma patients (which are different from). This treatment would not work due to their differing disease causes. A more personalized medicine could then be designed for their new, more precise diagnosis.

Personalized education is a dominant theme in HDA. Not all of the optional courses will be necessary for each student. Due to the wide range of next steps the students take, not all of the course weeks are not directly related where they end up. There may be material we do not cover that is relevant to where a student would like to end up. The program can only be personalized through structural procedure so much - the rest is up to you. Use the program as practice. After this course, you will have to personalize your growth completely. Start your personalized education journey with prep material - identify which areas you need to focus on and design a personal plan up to induction week.

Nothing learned is wasted. This course provides a multifaceted skill set that will help in any future career path. A project manager can better manage their team if they are familiar with programming fundamentals; A data scientist is more efficient if they know project management fundamentals; A data engineer benefits from understanding how to take a rigorous scientific approach; A research scientist needs to understand how to write scalable, reproducible code. We ask that you embrace every element of the course, even if it does not appear to directly relate to your future career. By becoming a multifaceted agile

thinker, you will stand out in whatever field you go into. Your task is develop a core competency in all of the subjects while identifying those that you would like to go deeper into for your next step.

Our students have a highly diverse range of backgrounds. There will always be some students with very little experience of each particular subject. Don't panic if you have a classmate with a PHD in molecular biology and you have never taken a science class. Each module in Term 1 starts with the basics which are built on in Term 2. Being more familiar with the material now will reduce any increases in difficulty throughout the program.

It is hard to learn anything completely the first time. The course is designed to introduce topics, reinforce their practical application with hands on exercises, and provide opportunities to explore specific topics in more depth with large scale real world problems. Seasoned researchers do not know every detail of every possible approach. They understand the theoretical underpinnings of the topic and have a broad understanding of what could be done. When designing a specific project, they will then go into more detail on the correct approach to use. Do not feel that you need to understand every detail of every approach in your courses. Adopt a consistent approach to learning and do not put pressure on yourself to instantly be an expert.

This is a challenging course that is designed to push you to discover your potential. While there may be moments that seem daunting, remember that other cohorts have successfully overcome the challenges and so can you. Here are a few quotes from students to keep in mind:

“One of the most valuable takeaways (personally) this year is just to be able to step into the unknown with confidence and pride” (A student about their excitement over starting their postgraduate role)

“There were times that I didn't think I would be able to complete the course. Having finished the program I feel like I can do anything” (A student who scored a Distinction)

“If you had told me what I was going to do in my summer project at the start of the course, I wouldn't have believed you.” (A student reflecting on their AI summer project having come into the program not knowing how to code)

2.1 Advice from the Teaching Team

Be proactive. This is your learning journey. We will provide the structure and support, but you need to take the initiative.

Be engaged. Use discussion board, come to Q&A with questions, engage with your teaching team, come to class and engage with your peers, and give the projects your best effort.

Be purposeful. Try to identify achievable goals. Have study sessions with an

achievable target in mind. Be conscious of what research decisions you are making.

Take risks. You have the chance to get detailed, instructive and honest feedback for maybe the last time in your life. Don't design projects to be as easy as possible or to be safe in order to get a high grade. Explore your research ideas and take on difficult projects so that the first time you have to struggle isn't when you have no help.

2.2 Advice from previous HDA students

We asked former students of the HDA course what advice they would give their past selves before starting the course. Their responses are below.

2.2.1 Before starting the course

- If you have questions, ask! Never be afraid of being curious!
- Don't be disheartened when you don't have a clue what's happening in R or python if you have zero prior coding experience - it's a steep learning curve but you'll pick it up. Coding is the sort of thing where there's a certain level of understanding beyond which everything just seems very understandable, and before that level it just seems super overwhelming. Just the ability to understand the syntax in R and the different nuances etc gives you the ability to understand and search every problem on stack overflow. You don't need to know a load of functions off by heart.
- If possible, do some of the data camp courses BEFORE starting the MSc
- Take an intro linear algebra course and don't just rely on the math refreshers

2.2.2 Throughout the course

- Organise and document everything during projects: scripts, data, notes etc. You will definitely revisit old code and old notes
- Be proactive, ask question and use the resources that are made available to you (the people around you and the lecturers)
- Keep your notes and code organised, I kept referring back to past tutorials in the project, and having a filing system from the beginning of the project made this so much quicker.
- Don't think you're the only one who doesn't understand something. There's a huge range of pre-existing skills and knowledge in the other people in the class and inevitably you'll know less about some stuff and more about other stuff than your pals
- Especially with the wide range of skill sets in the program, whenever I didn't get something I knew at least a quarter of our program did and had to fight my self consciousness to ask the question. Also lecturers are

really nice I would have asked a few more questions had I not been worried about looking like an idiot

- Get to know the people on the course and work with them – everyone has different strengths and will pick things up at different rates and it really helped this year working together to understand things.
- Capitalise on your classmates' knowledge! It's a nice symbiotic relationship
- Start projects earlier.
- Revise the second term lectures such that there are some baby steps leading to the big equations
- Summer quarter advice - make sure they have data you want to work with.
- Advice for the thesis: get your references and stuff organised and maybe don't wait too much before writing some stuff.

Chapter 3

Getting to Know Your Computer

There are two main sources of confusion to those relatively new to programming: understanding how to install open-source software and how to use the Terminal. While there are download instructions in this guide, you will need to understand your Computer Specifications so you can make the right selection for your machine. Additionally, understanding what the Terminal is and what it is doing offers a nice introduction to computer programming and can help make the interface feel more user friendly.

3.1 Computer Specifications

When installing a program on your computer, you are often presented with several versions of the software depending on your computer specifications.

3.1.1 Mac Computer Specifications

To determine the specifications of your Mac and find compatible software downloads, you should first identify your specific Mac model and then check the software's system requirements. You can find your Mac's specifications by going to About This Mac in the Apple menu. Then, compare these specifications with the requirements of the software you intend to download. The main thing to establish is whether you have a silicon or Intel processor.

Here is an example from the R Studio download page for Mac. Note that there are two versions - silicon and Intel - that you have to choose from.



Figure 3.1: Determining Silicon or Intel Processor

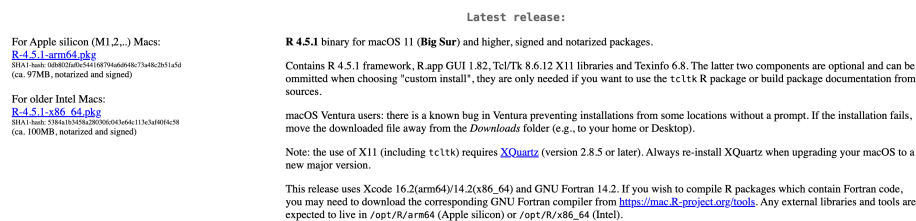


Figure 3.2: Example of Download Differences

3.1.2 Windows Computer Specifications

Most current software downloads are for Windows 10 or 11.

To check your Windows version, you can use either the Settings app or the “winver” command. In Settings, navigate to System > About. Alternatively, press the Windows key + R, type “winver”, and press Enter. The “About” section in Settings and the winver window will both display your Windows edition, version, and build number.

Important Edition and Versions:

Windows 10 (1903 and above) will be able to install everything that Windows 11 can, but earlier versions (1902 and below) or earlier editions (8.1 or earlier) cannot install the current version of most softwares. You will need to go into the archive of software downloads to find the version that works with your Windows edition. If you are having trouble with this, please let us know with the help form.

Software downloads will often come in three options which correspond to the different Windows system architectures: 32-bit, 64-bit, or ARM. To find out if your Windows PC has a 32-bit (x86), 64-bit (x64), or ARM architecture, navigate to Settings > System > About. Under “Device specifications,” the “System type” field will indicate whether your system is “x64-based processor,”

Windows Version
Windows 8.1 or earlier
Windows 10 (#1902 and below)
Windows 10 (#1903 and above)
Windows 11 *

Figure 3.3: Important Windows Edition and Versions

“ARM-based processor,” or “x86-based processor”.

3.1.3 Linux Computer Specifications

Linux is a family of open-source operating systems (OS) based on the Linux kernel, created by Linus Torvalds. It’s known for its versatility, security, and customization options, making it popular for everything from supercomputers to embedded systems. Essentially, it’s a free, open-source alternative to operating systems like Windows and macOS.

Typically those who use Linux have advanced technical skills. HDA does not teach Linux or offer technical support for it. However, all of the programming languages and tools that we use are compatible with Linux. HDA asks that you utilize a Windows or Mac operating system if you have the option. Please get in contact with the program using the help form above if you have to use Linux and are unable to complete this computer set up section.

3.2 The Black Box

Unix is a foundational operating system that provides the basic structure for running computer programs and managing hardware resources. You can access it with a Terminal. A Terminal is a text-based interface used to interact with a computer’s operating system. You might also hear it referred to as a CLI (Command Line Interface). Every Operating System — be it Windows, Linux or Mac has a terminal that users can use to interact with the OS.

The terminal is informally referred to as “The Black Box” given that it seems intimidating and overwhelming to those who haven’t used it, but have seen someone else interact with it.

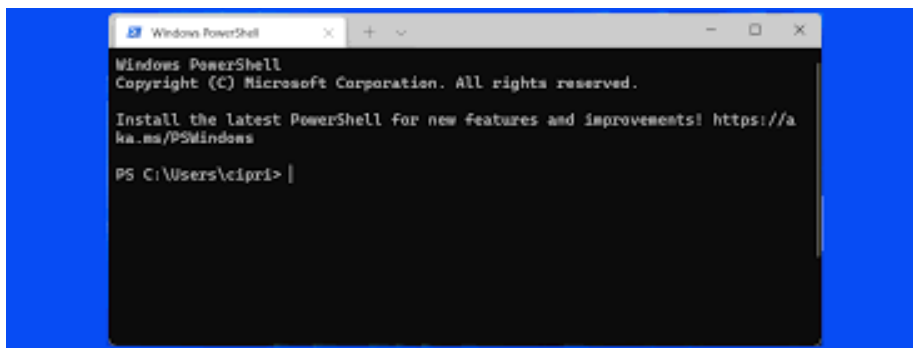


Figure 3.4: The Blackbox of the Terminal

The Terminal is commonly what is shown during “hacking” scenes in movies like the Matrix given that it will instantly impress any non-technical audience.

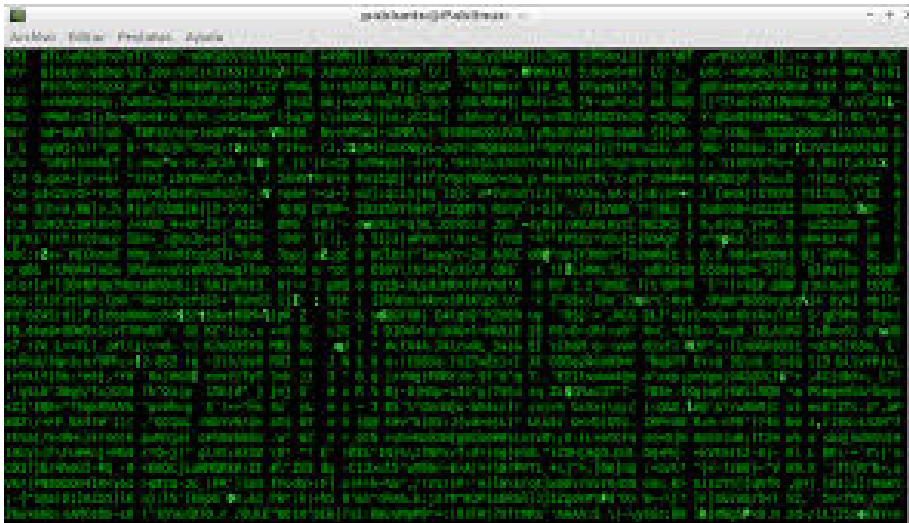


Figure 3.5: Common Depiction of Shell Scripting

3.2.1 Demystifying the Terminal

This Terminal is a software — essentially a translator that gives you access to a computer’s shell. Examples include the Command Prompt on Windows, and the Terminal on MacOS and Linux. When you open up a Terminal, you are able to type in text. Certain keywords will perform tasks on your computer.

The Shell is where the magic happens. After typing your command into the terminal, the shell interpretes that command, and executes it accordingly.

There are different types of shell, each with its own unique features. The Bourne-Again Shell known as Bash, is popular amongst Linux System Administrators. Bash is a shell used to manage files, run scripts for automation, and generally control your system.

HDA teaches Bash, but there is also Zsh (Mac), Fish, and PowerShell (Windows). The basic syntax is the same but each has different advanced capabilities for different tasks. For example, PowerShell is used for complex scripting, managing and automating Windows systems, managing virtual machines, and working with .NET Libraries. Bash offers a lightweight and relatively user-friendly interface that can accomplish every task you need in HDA while establishing a familiarity with the core features of shell scripting.

In the next section of the pre-course guide, you will use the Terminal to perform a few simple tasks to increase your comfort with the interface.

Chapter 4

Setting Up Your Computer

HDA uses the following programming languages: Bash, R, Python, SQL, Git, and Latex/Markdown. It can be overwhelming to learn so many new things in such a short time. We will use the set up of your computer to introduce the languages, tools we use to interact with them, and some key computing concepts. Don't feel that you need to complete the entire set up in one session.

It is important to note the distinction between HDA using the languages and HDA teaching the languages. The fundamentals of each language will be introduced, but the focus will be on the application and utility of the languages rather than improving programming skills. Students are provided a free subscription to Datacamp which they are expected to use to hone their programming skills.

If you have trouble installing any of these programs don't worry, you will revisit the installation of these in the induction week and across the first few weeks of the different modules.

4.1 Bash

Bash is a shell scripting language used to manage files, run scripts for automation, and generally control your system. There is no need to install Bash as every computer comes with a pre-built shell scripting language (like Bash).

Open the Terminal (remember it is called the Command Prompt on Windows) on your computer and type "pwd" and press enter. Then type "ls" and press enter.

```
(base) tjw19@IC-WHWY6GTJGT ~ % pwd
/Users/tjw19
(base) tjw19@IC-WHWY6GTJGT ~ % ls
Applications
Best_Results
Calibrations
Creative Cloud Files
DATA
Desktop
```

You have just told your computer to **print** the **working directory** (`pwd`) or tell you what folder you are currently in on your computer. We then told the computer to **list** (`ls`) the contents of that folder. Interacting directly with the operating system can be difficult, clunky, and intimidating for beginners. To overcome this issue, companies like Microsoft developed **Graphical User Interfaces** (GUI) to make interacting with the computer easier and more user friendly. Now you are able to double click icons on your desktop instead of needing to type into the terminal – vastly expanding the number of people who can use computers.

That is all you need to know about Bash for now as it will be covered in more detail in the course. The key takeaway is that a programming language allows you to interact with a program – here your operating system. There are different programming languages for different programs. GUIs make it easier and more user friendly to interact with key programs.

4.2 R

R is a domain-specific, statistical programming language. This means that it has been designed to allow users to perform mathematical and statistical analysis without needing advanced programming knowledge.

Install R and then come back here. (The link will take you to the “Installing R and R Studio” Chapter of this guide).

Open the Terminal. Type “R” and press enter. Your terminal should now be displaying R. Now type in “2+2” and press enter. You should get the answer of “4”. Type “print(‘Hello World’)” and press enter. Now type “pwd” and press enter. You get an “Error: object ‘pwd’ not found”. Type in “quit()” and press enter then “n” and enter when prompted.

You are now back in the Terminal. Type “2+2” and press enter. You get an error that the “command is not found”. Now type “print(‘Hello World’)”. You get an error that there is “unknown file attribute: H”. Now type “pwd” and press enter.

What has just happened? You have just written in two separate programming languages. Key commands in one language are not recognized in another lan-


```
R version 4.4.1 (2024-06-14) -- "Race for Your Life"
Copyright (C) 2024 The R Foundation for Statistical Computing
Platform: x86_64-apple-darwin20

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> 2+2
[1] 4
> print('Hello World')
[1] "Hello World"
> pwd
Error: object 'pwd' not found
> quit()
Save workspace image? [y/n/c]: n
```

Figure 4.1: Hello World in R

```
(base) tjw19@IC-WHWY6GTJGT ~ % 2+2
zsh: command not found: 2+2
(base) tjw19@IC-WHWY6GTJGT ~ % print('Hello World')
zsh: unknown file attribute: H
(base) tjw19@IC-WHWY6GTJGT ~ % pwd
/Users/tjw19
```

Figure 4.2: Hello World in Bash

guage. Bash is for interacting with operating systems so will have a lot of inbuilt commands to do so (like printing the user location). R is for statistical analysis so can handle numeric values and has inbuilt programming functions to make use easier (like print).

Like using Bash, this can be a bit clunky, difficult, and intimidating. All popular programming languages have at least one GUI to solve this issue. The most popular GUI for R is called R Studio.

4.2.1 Using a GUI

Install R Studio and then come back here. (The link will take you to the “Installing R and R Studio” Chapter of this guide).

Open R Studio. Type in “2+2” and press enter, then “print(‘Hello World’)” and press enter. You are typing in the console, which was the exact same thing that you accessed from the Terminal by typing in “R”.

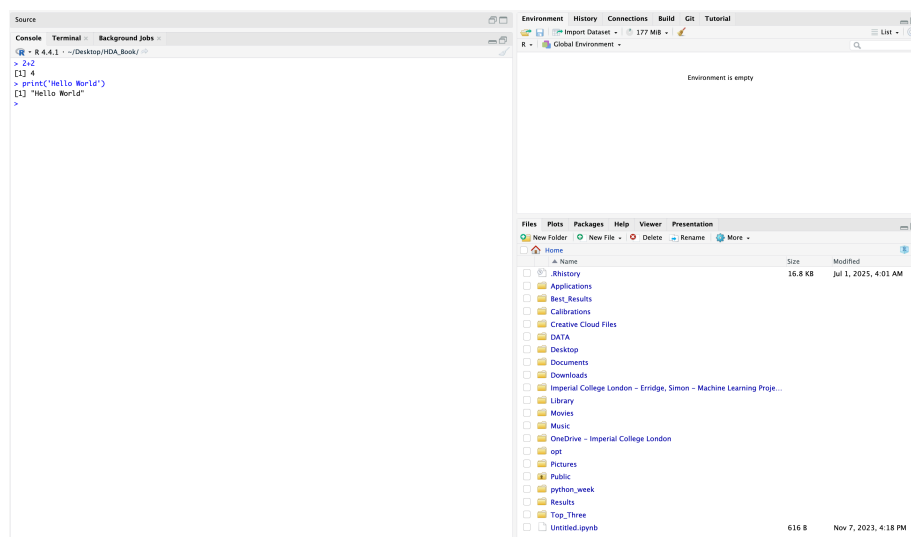


Figure 4.3: Hello World in Bash

Let’s see how this can be helpful. Create a new R Script by click on the option in the upper left of the program.

In the new script, type in “2+2” press enter then type “print(‘Hello World’)” and press enter. What has happened? Nothing since the script acts like a text document. Highlight both lines of code and then click “Run” in the upper right of the script window. Now you will get the output you expected.

If you wanted you could change part of the script easily and run it again. Or save it and send it to a friend. This is all that you need to know about the func-

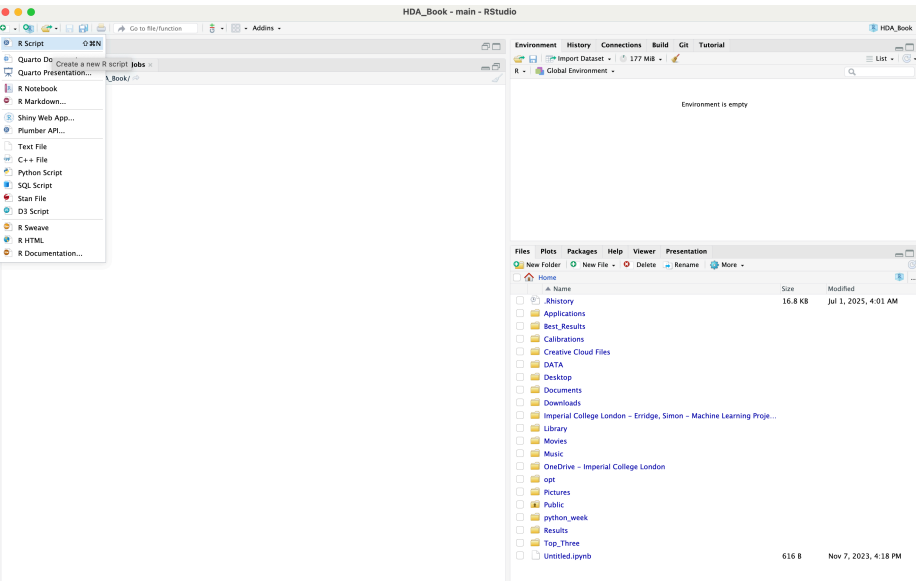


Figure 4.4: Hello World in Bash

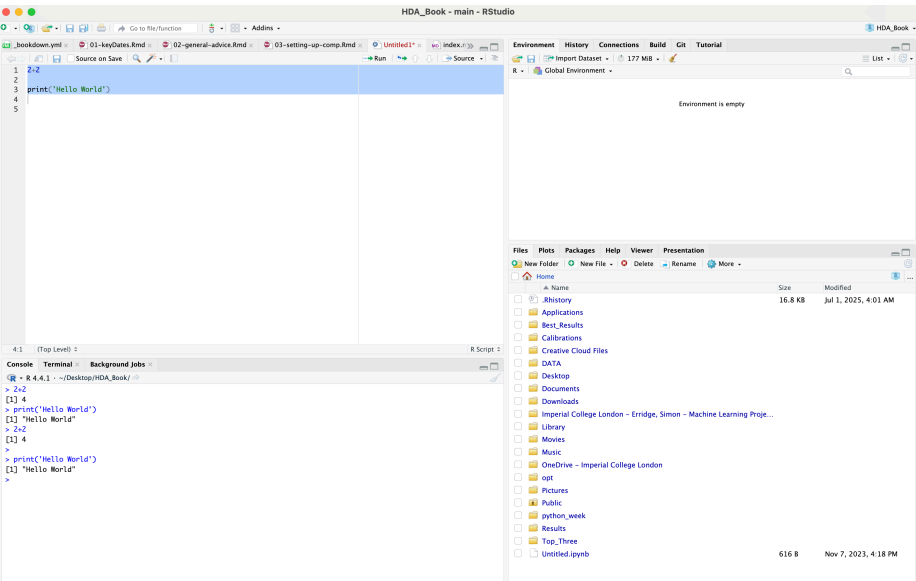


Figure 4.5: Hello World in Bash

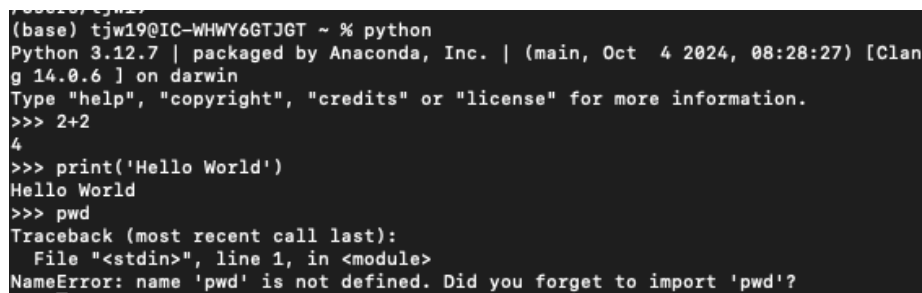
tionality of R Studio for now but it has many powerful features. For example: This book was written in R Studio (you can see the files open in the script pane in the picture above).

4.3 Python

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python’s simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance.

Install Python and then come back here. (The link will take you to the “Installing Python, Anaconda, and VSCode” Chapter of this guide).

Open your Terminal, type in “python” and press enter. Type in “2+2” and press enter, type in “print(‘Hello World’)” and press enter, type in “pwd” and press enter. Close your terminal window.



```
(base) tjw19@IC-WHWY6GTJGT ~ % python
Python 3.12.7 | packaged by Anaconda, Inc. | (main, Oct 4 2024, 08:28:27) [Clang 14.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 2+2
4
>>> print('Hello World')
Hello World
>>> pwd
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'pwd' is not defined. Did you forget to import 'pwd'?
```

Figure 4.6: Hello World in Python

What just happened? Same commands worked as in R, but the commands from bash did not work. Python and R have a lot of overlap. Generally, Python is less user friendly (but still accessible) but far more powerful and flexible. Most neural networks are written in Python. R is more user friendly, less powerful, but has more detailed and useful prebuilt statistical functions. It is important to know both in order to be able to use the right tool for the right job.

4.3.1 Python IDEs

There are more Python GUIs than there are R GUIs - but a lot of them are actually IDEs. An **I**ntegrated **D**evelopment **E**nvironment is a software application that provides a comprehensive environment for software development, often including a GUI. A GUI, on the other hand, is a type of user interface that allows users to interact with applications using visual elements like windows, icons, and

menus, rather than through text commands. IDEs are places to write code that offer some supportive features. Almost all IDEs provide syntax highlighting, debugging, and code completion.

R Studio is technically an IDE, but is commonly referred to as a GUI. This is because users do not use R to write software. Python is primarily used to develop software, so the associated programs are referred to as IDEs. The most common Python IDEs are Jupyter Notebook, Pycharm, and VSCode.

Install Python IDEs and then come back here. (The link will take you to the “Installing Python, Anaconda, and VSCode” Chapter of this guide).

4.3.2 Jupyter Notebook Exercise

Open Anaconda and then Jupyter Notebook. When the program opens, click on “New” and then the “Python 3 (ipykernel)” option:

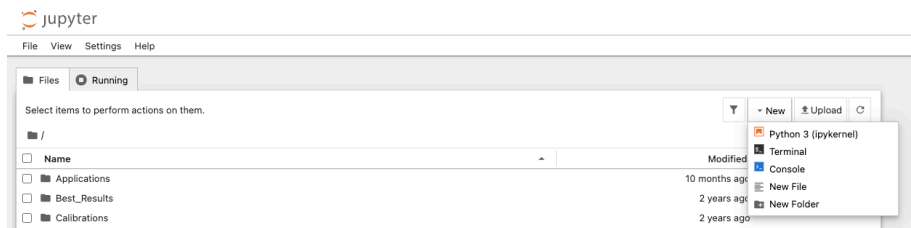


Figure 4.7: Opening Jupyter Notebook

The Program should look like this:

The “box” you can see is called a code cell. Create another code cell below the current one by pressing the button on the right hand side of the code cell that corresponds to adding a cell below:

Type the same two lines in each of the boxes: “#adding some numbers” and “print(2+2)” (without the quotation marks). Click on the second box and change the option from Code to Markdown in the ribbon at the top of the document:

Run both of the code cells:

What happened? Does yours look like this?

Jupyter notebook allows you to write Python code or “markdown”. Markdown is another programming language - but don’t worry, you don’t have to learn it in any detail! It just allows you to type text that is nicely formatted or code that can display the output. This allows for nicely formatted reports and documents. For example: each chapter in this book is written in R Markdown (which uses markdown as well).

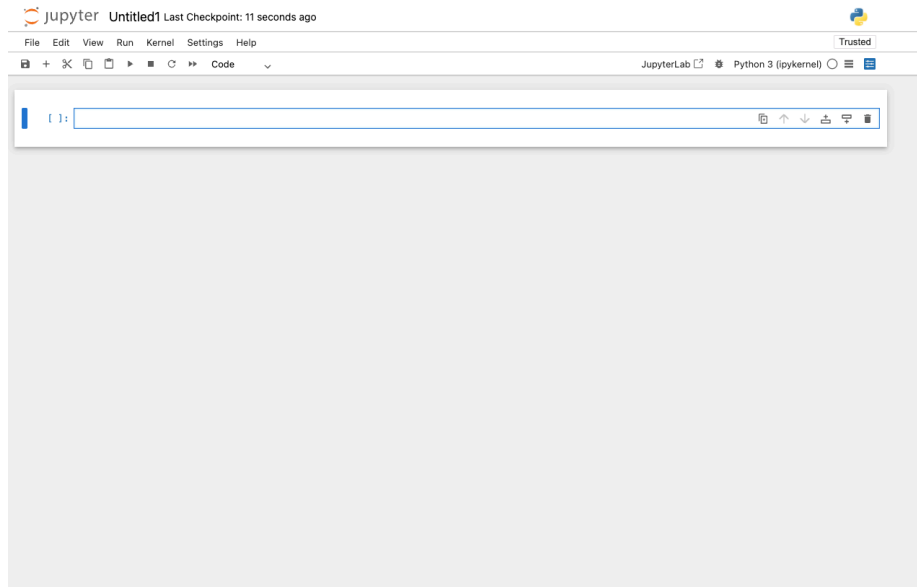


Figure 4.8: What a Jupyter Notebook Looks like

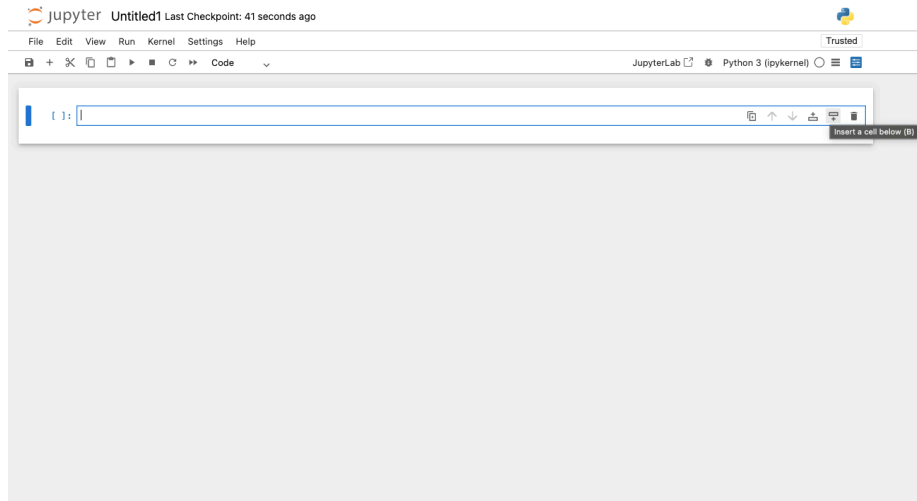


Figure 4.9: Adding More Code Cells

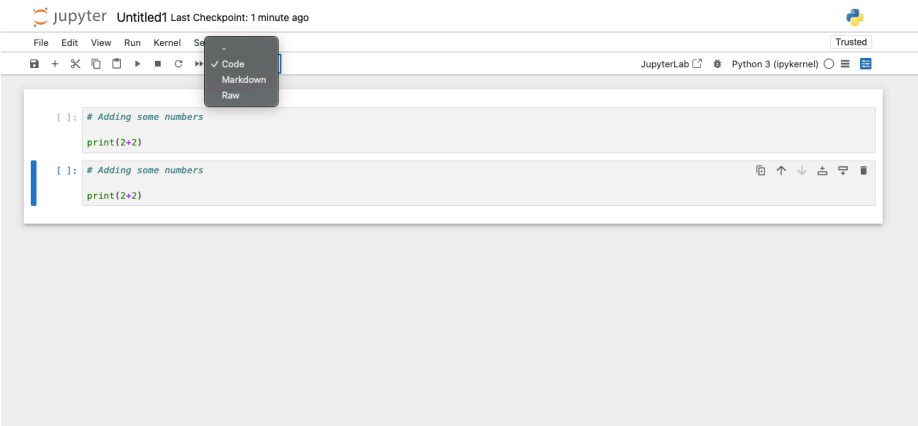


Figure 4.10: What to Input

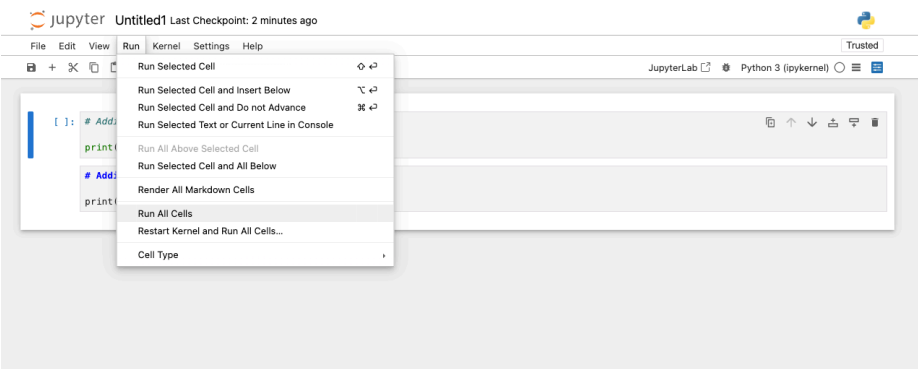


Figure 4.11: How to Run the Entire Script

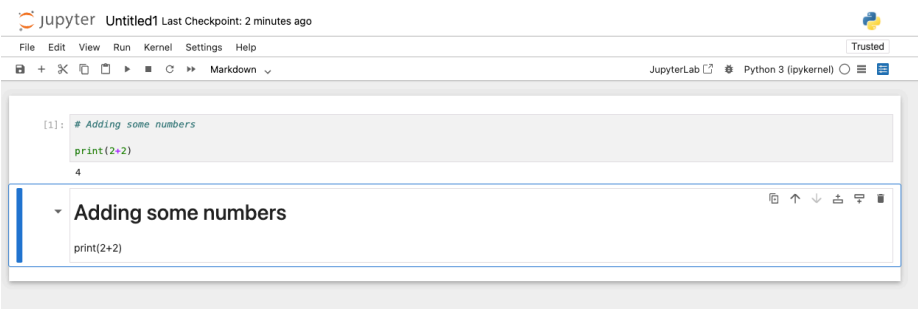


Figure 4.12: Jupyter Output

4.4 SQL

Structured query language (SQL) is a domain-specific, standardized programming language for storing and processing information in a relational database. It is the best language for collecting, manipulating, and querying large amounts of data. Most search engines (Google), plotting tools (Power BI), and platforms (Salesforce) turn your commands in their user interface (a GUI) into a SQL query that it gives to its database.

There are multiple different dialects of SQL, much like there are multiple different dialects of shell scripting languages. The different dialects of SQL (MySQL, SQLite, SQL Server, Oracle, etc.) all have the same syntax for most of their commands. Different dialects have different uses or additional capabilities for specific use cases. Learning in one SQL dialect will allow you to quickly pick up most of the other ones. HDA teaches MySQL.

Install MySQL and then come back here. (The link will take you to the “Installing MySQL and DBeaver” Chapter of this guide).

4.4.1 DBeaver

DBeaver is free and open source universal database tool for developers and database administrators. It is a SQL client software application and a database administration tool. It works with any version of SQL and with any database - whether on your computer or on the internet.

Install DBeaver and then come back here.

4.5 Git

Git is a distributed version control system that tracks versions of files. It is often used to control source code by programmers who are developing software collaboratively. Git. A command-line session showing repository creation, addition of a file, and remote synchronization.

GitHub is a cloud-based platform where you can store, share, and work together with others to write code. Storing your code in a “repository” on GitHub allows you to: Showcase or share your work. Track and manage changes to your code over time. Let others review your code, and make suggestions to improve it.

GitHub is widely considered an industry standard for software development and version control. It’s the dominant platform for hosting and collaborating on code, used by millions of developers and organizations globally. Over 90% of Fortune 100 companies use GitHub.

Install Git and then Make a Github Account

You will be introduced to Git and Github in the Translational Data Science module of the programme, so you do not need to be a master now.

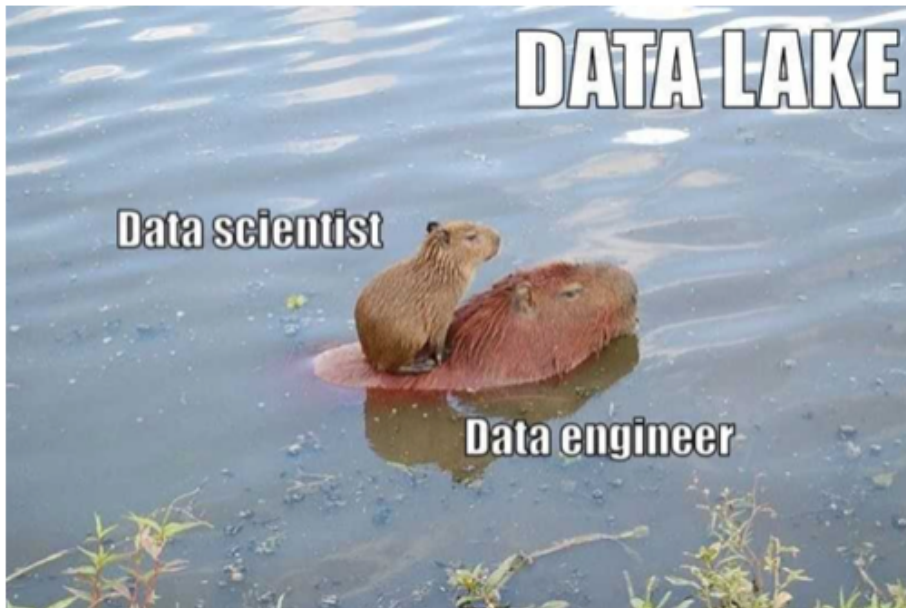


Figure 4.13: Downloading R from the CRAN

4.6 Other Programming Languages

A Turing-complete language (also called a universal language) is one where you can compute anything that any other computational method can compute. In other words, a language that's non-universal—or Turing incomplete—has some limits on the set of things that it can compute

Many programming languages are Turing complete, meaning they can perform any computation that a Turing machine can. However, some languages are specifically designed to be non-Turing complete, often for reasons of safety, predictability, or domain-specificity. Examples of non-Turing complete languages include: regular expressions (in their standard mathematical definition), HTML, CSS, and some versions of SQL

So why are there so many programming languages? They are each optimized to make different popular tasks easier. While the syntax varies drastically between languages, the core fundamentals are the same. Learning the most popular and user friendly languages (R, Python, and SQL) is a great way to start your programming journey and may be all that you end up needing. It should be noted that this course does not teach or require computer science skills, front-end development, or back-end integration.

Chapter 5

Prep Material

HDA builds on quantitative and computational fundamentals to apply big data analysis to the medium of environmental and molecular epidemiology. Using Health Data not only provides expertise in an evergreen field, but also offers the biggest of big data challenges to hone transferrable skills in.

Each student enters with a different level of expertise in these key areas. Due to this disparate skill set, they need to adopt a personalized approach to their education. HDA encourages each student to identify which areas they are least familiar with and put more effort into leveling up those skills.

A reminder that it is not necessary to be an expert in any of these topics at the start of the program. Applying a consistent effort (30-60 minutes a day) in a targeted manner will help make the program easier and complex topics less daunting.

5.1 Molecular Biology

Several of the courses in the program apply transferrable techniques to omics data. Omics data refers to the comprehensive datasets generated in “omics” fields of biology, which study the totality of molecules and processes within a biological system. These fields, like genomics, transcriptomics, and proteomics, aim to analyze the complete sets of DNA, RNA, proteins, and other molecules. Omics data provides insights into the complex interactions within cells and organisms, contributing to a deeper understanding of biological systems.

Some of the former students and teaching staff on the course have put together a nice cheat sheet with some of the concepts you will need to pickup throughout the course. You will review these in Molecular Epidemiology or in our optional Molecular Biology classes but this is a great starting point.

5.2 Quantitative Skills

HDA focuses on the application and interpretation of advanced methodologies. You do not need to be a math expert to succeed in the course. However, understanding core quantitative concepts will make the advanced material more approachable.

For example, in Natural Language Processing (NLP), cosine similarity is a metric used to determine how similar two text documents are to each other, regardless of their size. It is calculated by representing the documents as vectors and calculating the dot product of the vectors divided by the magnitude of each vector. This can be very confusing if you aren't familiar with vectors in the first place. Another example, Stochastic gradient descent is an iterative method for optimizing an objective function with suitable smoothness properties. It can be regarded as a stochastic approximation of gradient descent optimization, since it replaces the actual gradient by an estimate. This can be really confusing if you don't know what a gradient or optimization is.

Understanding a few core quantitative fundamentals will make it much easier to understand how and why certain methods should be used.

5.2.1 Math

What do self driving cars, setting the quota in a fishery, determining a product launch campaign, and finding novel causes of disease all have in common? They are solved by constrained optimization. Statistics employs exact optimizations to generate solutions. Different machine learning algorithms change the constraints, optimizer, or both. The follow external videos provide a general introduction to the concept of optimization. You will not need to perform any complex math in the course, but you will utilize methods that do so. Understanding what they are doing on a conceptual level will make the methods more approachable.

Intro to Optimization

Least Squares Optimization

Convex vs. Non Convex Optimization

Visualizing Convex Optimization

5.2.2 Linear Algebra

Most numeric optimizers and programming languages utilize linear algebra. The following external videos provide a good conceptual overview and visual representation of the key concepts you will see in the program.

Linear Algebra Overview

Note: The above video has an informal editing style and delivery. However, it is an excellent overview of a large number of complex topics in a digestible format.

Detailed Linear Algebra

Note: These videos are longer and in greater detail. HDA recommend watching videos 1-6 and 9 in the above playlist.

5.2.3 Statistics

Statistics underpins most analytical fields - epidemiology, machine learning, AI, finance, and more. It is important to be precise, understand the limitations of the approaches, and understand how to interpret the outcomes. The following videos provide an excellent informal introduction to statistics. The statistics course in HDA will provide a more rigorous, formal and precise statistical foundation.

Statistics Overview

Detailed Statistics Overview

5.3 Programming

The best way to get better at programming is by practicing. As an Imperial student, you will have free access to Datacamp. HDA recommends the following courses if you are interested in practicing your skills before the program: Introduction to R, Introduction to Python, Introduction to SQL, Foundations of Git, and Introduction to shell. These courses follow on from the introduction to the programming languages covered in HDA provided in this guide. To access the DataCamp courses, you need to use your Imperial email account.

5.3.1 Other Practice Platforms

There are other platforms that you can use to practice your programming skills for free. The most popular ones include Leetcode, HackerRank, CodeSignal, Codewars, and AlgoExpert. These exercises are more focused on computer science skills than on data science. These skills are not needed to succeed in HDA and not relevant to all next steps. If you decide you want to pursue a programming heavy career option, then it is a good thing to start practicing every day. HDA does not recommend starting with material not strictly relevant to the program, but would advise those that are interested to start looking into those options in Term 2.

Kaggle is a data science competition platform and online community for data scientists and machine learning practitioners under Google LLC. It has a variety of free courses and past competitions with solutions. While an excellent resource,

HDA would recommend establishing a base familiarity with the other material in this guide before exploring Kaggle.

5.4 Fun Readings

Not all learning needs to be technical and detailed. It can be useful to read interesting short articles or books to see how the technical skills you are learning could be applied.

5.4.1 Blogs

This is a collection of some of the most popular blogs in relevant spaces to the program. You do not need to follow all of them, or any of them. HDA recommends browsing a few from each category to begin to understand where your interests may lie.

Data Science and ML

- <https://www.kdnuggets.com/>
- <https://towardsdatascience.com/>
- <https://www.analyticsvidhya.com/blog/>

AI

- <https://bair.berkeley.edu/blog/>
- <https://news.mit.edu/topic/artificial-intelligence2>

Programming

- <https://news.ycombinator.com/>
- <https://blog.tensorflow.org/>
- <https://www.mygreatlearning.com/blog/>

Biotech

- https://www.sciencedaily.com/news/plants_animals/biotechnology/
- <https://www.nature.com/nbt/>
- <https://www.fiercebiotech.com/>
- <https://www.fiercepharma.com/>

5.4.2 Books

While some people prefer to have physical books, you can also download PDF versions of most books for free from the internet in a safe, legal and ethical manner.

Where to Download PDFs

Here are a few books that introduce key concepts of the program that are favourites of the staff. Note: None of the staff have any association with the authors or publishers of these books and do not endorse them further than being entertaining reads.

What is a P Value Anyway?

Signal through the Noise: Why so many prediction fail

Junk DNA

Chapter 6

Course Preview

Module previews to get familiar with the courses before learning the in-depth details. Please note that the below may be subject to minor changes.

Term 1

Introduction to Statistical Thinking and Data Analysis

The aims of this module are to help students to:

- develop the knowledge and skills required to conduct and interpret statistical analyses of epidemiological and health data;
- understand and use different types of data, study designs, and statistical tools - including software - suitable for applied research and practice.

By the end of this module, students should be able to:

1. Demonstrate the ability to interpret statistical inference, sampling from a population, confidence intervals and hypothesis testing.
2. Apply and contextualise statistical methods such as t-tests, linear regression, logistic regression and survival, in the epidemiological setting.
3. Assess and verify the assumptions that apply to each method and the implications of violating assumptions for validity of analysis.
4. Visualise, manipulate, and analyse data using R.
5. Prepare and analyse real world data sets, addressing the implications of sampling methods and missing data.
6. Interpret and communicate, both orally and in writing, findings arising from the application of these statistical models to relevant audiences.

Module content

- Principles of Inference, Sampling, Normal Distribution

- Confidence intervals, Hypothesis testing, and p-values
- Linear regression
- Multiple linear regression and model building
- Binary outcomes, comparing proportions, chi-squared tests
- Logistic regression
- Longitudinal data, Poisson regression
- Survival analysis and the Cox proportional hazards model
- Statistical modelling and maximum likelihood
- Bayesian Inference, Missing data
- Study design, Sample size calculation

Principles and Methods of Epidemiology

Principles and Methods of Epidemiology is a core module that ensures students will be familiar with the core concepts of epidemiology and acquire the skills necessary to describe, analyse, interpret and appraise epidemiological studies. Further modules and projects require such knowledge, and a good grasp of these basics is thus essential for successful completion of the degree.

By the end of this module, students should be able to:

1. Choose, calculate, and interpret appropriate measures of disease frequency, measures of association and measures of attributable risk for given scenarios;
2. Describe and contrast strengths and weaknesses of different epidemiological study designs and interpret the results of meta-analyses;
3. Recognise and discuss different types of biases that may occur in epidemiological studies, and describe strategies that can be used to reduce such biases;
4. Evaluate whether confounding and/or effect modification may have affected a given exposure-outcome relationship, and describe and apply techniques to deal with this at the design and analysis level;
5. Critique a published paper, interpret its findings considering its strengths and limitations, and assess whether an association may be causal.

Module content

The module covers key epidemiological concepts including:

- measures of frequency and association;
- attributable risk;
- disease detection;
- screening;
- population and high-risk approaches to prevention;
- study designs;
- study validity;
- selection bias;
- information bias;

- confounding;
- effect modification; and,
- causal inference.

Molecular Epidemiology

The aims of this module are to enable you to:

- apply the different techniques taught to their respective data types and use them in large scale epidemiologic studies to analyse and interpret the data;
- understand the analytical methods and challenges you may face in applying and interpreting the results of these molecular epidemiology studies;
- read and interpret the literature based on molecular epidemiologic studies.

At the end of this module, students should be able to:

1. Classify omics data according to their underlying data generating process.
2. Evaluate, assess, recommend and implement quality control checks on various omics data.
3. Analyse omics data using relevant software and statistical packages.
4. Interpret findings from omics data analysis using different web tools.
5. Critically appraise papers describing findings using the molecular epidemiology approach.

Module content

- Multi omics
- Introduction to genetic epidemiology
- Genome-wide association studies
- Mendelian randomisation approach
- Epi-genomics
- Metabolomics

Clinical Data Management

This module aims to familiarise students with the complexity and diversity of clinical/medical data and to enable students to use and query clinical databases that are SQL based. The module will introduce key features and requirements attached to clinical data and will present methodologies and standards used to design and exploit databases. Specifically, the module will provide an overview of:

- Legal/ethical/security issues that are inherent to clinical data;
- Established methodologies, standards and platforms used to create efficient, re-usable and portable databases;
- Efficient programming languages to build, maintain and interrogate databases;

- An introduction to simulation, which is crucial for instances when data sharing is not possible;
- An introduction to electronic health records.

By the end of this module, students will be able to:

1. Identify and explain the different steps involved in the problem solving and data management life cycle;
2. Create, maintain and query relational databases using SQL;
3. Assess if a dataset or database is compliant to security, ethical and legal requirements;
4. Implement and compare different methods to anonymise and encrypt data;
5. Evaluate the anonymity of identifiers and classify them as sensitive, direct or indirect identifiers and measure the k-anonymity of a dataset;
6. Produce a pseudo-simulated or fully simulated dataset using available R or Python libraries;
7. Determine and argue the advantages and disadvantages of using electronic health records and explain how they are being used in computational health research;
8. Produce reproducible code and write analyses reports that communicates their investigations in a scientific and professional manner.

Module content

- Introduction to clinical data management
- Database Design
- Introduction to SQL
- Applications of SQL
- Introduction to Python
- Data Sharing and Data Privacy
- Data Anonymisation
- Introduction to Simulation
- Electronic Health Record Management and Analysis

Term 1 and Term 2

Translational Data Sciences

The Translational Data Sciences module aims to guide students in the process of producing a publishable piece of research by improving their team working, communication, presentation and research skills. Based on real data sets and supervised by field experts, students will have to apply methods learned on the other modules to solve a relevant problem on a real data set. This will involve going through the following steps to produce, by the end of the module, one oral presentation, and one draft publication, which will be peer reviewed:

- Data exploration;
- Literature search and research question definition;

- Design of a sound and realistic analytical plan;
- Implementation of the analytical plan;
- Revise and refine the analytical plan;
- Results visualisation and exploitation;
- Critical appraisal of the results and perspectives definition.

At the end of this module, students will be able to:

1. Design and implement an analytical research plan to analyse huge datasets applying statistical, machine learning and data management methods;
2. Manage the complexities of processing and analysing secure and large datasets;
3. Produce original results and contextualise these in the broader medical and health analytics field;
4. Produce a written report and oral presentation that communicates research succinctly and effectively to a scientific audience;
5. Appraise and critically assess their own research and that of peers;
6. Prepare a response to reviewers that makes amendments to research or justifies the choices made.

Module content

In Part I, the module will first provide students with some basic computational tools and skills that will help them throughout the term with other modules and those of Term 2: R, Unix, high-performance computing skills and visualisation tools. In Part II, the module will call upon the methods studied during the other modules of Term 2 and will require immediate solicitation of the knowledge freshly acquired to successfully implement them in real-time for the in-depth analyses of real data in relation to a real research question. Students will work in groups throughout the entire term on their group project.

Overall, this module will place students in the situation of researchers from the field of data science where cutting-edge methods are needed to fully exploit complex existing datasets. Projects proposed will borrow from the following (non-exhaustive) fields:

- Computational epidemiology;
- Bioinformatics;
- Computational statistics.

Term 2

Computational Epidemiology

The aim of Computational Epidemiology is to familiarise students with some key topics in computational biology, as a pivotal research theme within the School of Public Health. The module will introduce some of the issues faced while analysing complex data sets in advanced epidemiology. From these illustrations,

methodological developments to address the resulting technical/computational challenges will be described, assessed and compared. Each of these established methods will be associated to a practical session during which students will implement the methods in R.

By the end of the this module students will be able to:

1. Describe and define the main type of OMIC data: what they measure, their statistical features and biological relevance and the treatment each of these data require;
2. Implement, analyse and evaluate the different computational epidemiology models taught on the module
3. Interpret and contextualise in the realm of the application or study, the results arising from the implementation of the models;
4. Generate and interpret post modelling analyses using gene enrichment analyses and efficient visualisation tools;
5. Specify the challenges and formulate rigorous research questions for OMICs integration and the ongoing methodological developments used to perform these analyses;
6. Apply and interpret advanced modelling techniques including network models and Bayesian variable selection techniques;
7. Conduct OMIC profiling and integration on real data, going from data processing to results exploitation and to reporting of results/findings in a rigorous scientific format (oral and written presentation).

Module content

- Univariate models for OMICs profiling
- Penalised models for OMICs data analyses
- Risk modelling using high dimensional data
- Partial Least Squares
- OMICs Integration
- Introduction to network modelling and causal modelling
- Stability Selection
- Introduction to Bayesian Inference
- Bayesian Variable Selection
- Hidden Markov Models

Machine Learning

This module aims to familiarise students with the principles of machine learning (ML) and the most common ML methods (i.e., algorithms) used in health data analytics. The module will start with the definition of machine learning and its role in data analytics, compared to traditional statistical approaches. The module will then provide an overview of a series of algorithms in three layers of knowledge: (a) a plain language overview and a step-by-step explanation of the method, the rationale for the method, the appropriate use and interpretation of the outcome; (b) a detailed computational derivation with mathematical theory,

when appropriate; (c) practical hands-on work with real data applications of the method.

By the end of this module, students will be able to:

1. Argue the main aims of machine learning (ML) and compare and contrast the more commonly used families of ML methods
2. Explain the added value of ML in comparison to classical statistical inference giving examples;
3. Select and justify the most appropriate ML algorithm(s) to be used on a given dataset and problem;
4. Design and implement an analytical plan that uses the following ML algorithms:
 - decision trees, random forests and XGBoost,
 - at least three different clustering algorithms,
 - Artificial Neural Networks (ANN),
 - Convolutional Neural Networks (CNN),
 - Recurrent Neural Networks (RNN),
 - Natural Language Processing (NLP);
5. Interpret results arising from the application of ML models in the ML context;
6. Analyse real data using ML methods and produce an oral report that communicates findings in a rigorous scientific format using the terminology of the field.

AI for Health Sciences

The Artificial Intelligence for Health Sciences (AIHS) module will equip students with a comprehensive and practice-driven understanding of Artificial Intelligence (AI) approaches and applications for health sciences. The module covers a range of AI methods and illustrates their applicability to health data. The module will begin with a foundational overview of AI and its growing role in health sciences, differentiating it from statistical and Machine Learning approaches and exploring its transformative potential and limitations. A particular focus will be made on the development of rigorous and reproducible approaches and on the careful interpretation of the generated results.

By the end of the AIHS module, students will be able to:

- Critically appraise the use of AI, relative to statistical and machine learning approaches, in health sciences and,
- Using Bayesian inference, interpret uncertainty in the application of AI models.
- Evaluate the use of the main different AI techniques including (convolutional, recurrent and graph neural networks, deep generative models in analysing and integrating health data

- Make recommendations regarding the most appropriate AI algorithm and/or architecture for a given health data analytics research question
- Explain, and make recommendations regarding, the ethical issues raised by the use of AI models in health applications

Module content

- Definitions: AI vs Statistics and ML
- Supervised, unsupervised and reinforcement learning
- Feature engineering and deep learning
- Neural networks, convolutional and recurrent neural networks
- Natural Language Processing and LLMs
- Graph Neural Networks
- Bayesian inference, and Uncertainty in DL
- Deep generative models (VAEs, NF, DDP)
- Interpretability, fairness, ethics

Term 3**Research Project**

The research project is an exciting and central component to the MSc in Health Data Analytics and Machine Learning. It occurs throughout all of Term 3 and students are expected to work full-time on their research project. It provides an education in research and scientific ethos and methods. It also gives training in research techniques. With the guidance of their project supervisors, students are expected to apply many of the skills acquired throughout terms 1 and 2 to help them complete their research project. In some instances, new methods will need to be learned and applied. Integral to the project is the preparation of the paper style report and the oral examination on which they will be assessed.

At the end of the research project, students will be able to:

1. Formulate and propose an original and relevant research questions from a complex data set.
2. Explore and analyse a complex data set to assess its quality and its potential.
3. Deploy/exploit databases under all necessary quality/security/privacy standards.
4. Design and implement a complex analytical plan using methods borrowing from OMICs profiling and Machine Learning approaches.
5. Critically appraise results and adapt the analytical strategy accordingly.
6. Fully interpret results from the data analysis, including the production of efficient visualisation of the results.
7. Write and present a scientific report which effectively communicates the research conducted throughout the project.

Module content

Students will choose a project they are interested in from the following (non-exhaustive list of) fields:

- Molecular Epidemiology (multi-OMICS)
- Cancer Epidemiology
- Nutritional Epidemiology
- Global Epidemiology
- Infectious Disease
- Database Management and exploitation
- Population Sciences
- Exposome Sciences
- Big data from industrial partners

Each of the proposed projects will feature complex and high dimensional (sets of) data, some of which may be sourced and supplied by industry partners. As taught in the “Translational Data Sciences” module, each student will have (under the guidance of the Project supervisor) to go through all the process of writing a scientific article:

1. Perform a thorough literature review in the field and Identify an original research question. This will be reported in the background of the report and oral presentation.
2. Design a sound analytical plan, also presented and assessed in the report and presentation.
3. Implement using the most appropriate statistical and machine learning approaches to address the research question.
4. Critically appraise the preliminary results.
5. Refine the models and implement series of sensitivity analyses.
6. Conclude on the results and confront /discuss these findings against the literature”

Chapter 7

Reading List

Each module has an accompanying reading list. Most of the material is optional - either to help further your understanding or provide examples of more advanced applications for those that are interested. It is key that you keep your personal education journey in mind when approaching the optional material. Not all of it may be relevant or necessary for you. Additionally, you will have access to the library system for up to three years after graduating. This means that you can always return to material that becomes more relevant to your next step at a later date.

Most of the readings are articles that are available on Leganto (Imperial's internal reading list system). You will have digital access to all of those articles. Two courses have textbooks that accompany the in class material. These can be borrowed from the school library (subject to availability), purchased, or downloaded as a PDF. A remind that you can download PDF versions of most books for free from the internet in a safe, legal and ethical manner.

Where to Download PDFs

7.1 Statistics

- **Essential Medical Statistics.** Kirkwood B and Sterne J (2003) (2nd ed). Blackwell Science Ltd. <https://www.amazon.co.uk/Essential-Medical-Statistics-Essentials-Kirkwood/dp/0865428719> Chapters 2–5 will support what you learn in the term 1 statistics module.
- **An Introduction to Medical Statistics.** Bland M (2015) (4th ed). Oxford University Press. An alternative excellent standard text book. This includes multiple choice and other questions at the end of chapters, which may be useful for revision purposes. <http://www-users.york.ac.uk/~mb55/intro/introcon.htm> contains useful additional material. Chapters 4–7 will support what you learn in the term 1 statistics module.

7.2 Programming

There are no accompanying programming textbooks for HDA. For those interested in having a textbook, the following are free PDFs to download which contain material covering most of the key programming concepts.

Hands-on programming with R, Garrett Grolemon

R for Data Science, 2e, Hadley Wickham, Mine Çetinkaya-Rundel and Garrett Grolemon

Intro to Python Programming

Note: HDA does not cover recursion as it is more relevant to computer science than large scale data analysis.

SQL

7.3 Epidemiology

Basic Epidemiology Bonita et al. Available free of charge from: https://apps.who.int/iris/bitstream/handle/10665/43541/9241547073_eng.pdf?sequence=1

Recommended reading

These books are not covered in the course but have been supplied in the reading list by the module team for those interested in Epidemiology.

- **Oxford Handbook of Epidemiology for Clinicians.** Ward et al. <https://oxfordmedicine.com/view/10.1093/med/9780198529880.001.0001/med-9780198529880>
- **Epidemiology(Fifth edition.).** Szklo, M. and Nieto, F.J. (2014). Burlington, Massachusetts, Jones & Bartlett Learning (approx. £93). Available to purchase from: https://blackwells.co.uk/bookshop/product/9781449604691?gclid=Cj0AQ2g9wGEkLUMAkkoIY_FO62AVEVFj2neeE93lnZKq-zCsJF7IuAp65kaAgAKEALw_wcB
- **Gordis Epidemiology.** Gordis, L. (2014). Philadelphia, PA: Elsevier Saunders (approx. £35). Available to purchase from: <https://www.ukelsevierhealth.com/epidemiology-9781455737338.html>
- **Oxford Handbook of Epidemiology for Clinicians (Oxford Handbooks Series)** Ward H, Toledano M, Shaddick G, Davies B, Elliott P (2012) . Oxford University Press, Oxford UK. <https://www.amazon.co.uk/Oxford-Handbook-Epidemiology-Clinicians-Handbooks-ebook/dp/B00C3XJMQ0>.
- **Essential epidemiology: an introduction for students and health professionals.** Webb, P., Bain, C. and Page, A., 2016. . Cambridge University Press. <https://www.amazon.co.uk/Essential-Epidemiology-Introduction-Students-Professionals/dp/0521177316>

Chapter 8

FAQ

The HDA-ML team has put together a list of FAQs to address questions students commonly have. Please reach out to one of the HDA staff members if you have a query that is not addressed in this section. [HDA Contact Information](#)

1. Do you have any suggestions for finding accommodation in London?

When searching for accommodation, take note of the campus where most of your classes will be delivered. The MSc Health Data Analytics and Machine Learning course is based at the White City Campus in West London. However, some of your classes might take place at the Charing Cross Campus in Hammersmith and in the South Kensington Campus but the majority will be held at White City in the School of Public Health.

For additional information on accommodation, visit the [Imperial College Accommodation website](#).

2. How much money will I need to live and study in London for a year?

Please visit the [Imperial Living in London website](#) for detailed information.

You will need to travel between campuses by bus, bicycle or underground train. Some journeys may be convenient on foot.

3. What is my tuition fee status?

Please visit the Imperial Fees and Funding website for the most up to date information on tuition fees.

4. How and when should I send official documents (e.g. degree certificates)?

Official documents need to be sent to Registry as soon as they are requested . Please check My Imperial portal for instructions on how to send these. All outstanding conditions of your offer will have to be met before you can be offered an unconditional place and register as a student. Registering as a student is a pre-requisite for joining the course. For any questions regarding your documents and outstanding conditions, you can contact Registry directly

5. When does the course start?

The MSc Health Data Analytics and Machine Learning Course starts on 27th of September 2026. Further details of the first week of term (i.e. ‘Induction Week’) will be sent via email.

For international students, Imperial College London organises some orientation events prior to the official start of the course. Please visit the International Students Webpages for further information.

6. I’m an international student – what else should I consider or do before my arrival?

Please bear in mind that you may need a visa and vaccinations before your arrival. If you think you will not be able to obtain the visa in time for the start of the course please contact the Postgraduate Education Coordinator, Ms Jo Tite: j.tite at imperial.ac.uk

You may also want to familiarise yourself with information on how to open a bank account, and how to find a general medical practitioner (GP) after your arrival. Options for GP services include the Imperial College Health Centre in South Kensington or other services closer to your chosen accommodation.

Please also make active use of the general information resources for international students at Imperial.

7. Where does teaching take place?

Teaching takes place most at the White City Campus in the School of Public Health. Occasionally you might have some classes in South Kensington and in Hammersmith. This will depend on the modules taken.

Please familiarise yourself with Imperial campuses and maps

Note: The name of the campus may not correspond to the name of the nearest Underground station. For example, the Charing Cross campus is not near the Charing Cross tube station (but is between Hammersmith and Baron's Court stations), and the Hammersmith campus is not near Hammersmith station.

8. Do you have a reading list of books to prepare for the course?

We encourage you to do some general preparatory reading; we have attached a list of key texts in the previous sections. However, we do not anticipate prior learning in the subject matters, and the modules are taught starting from a basic level – so there will be adequate opportunities to learn the subjects even if you have never taken these modules beforehand. During term time, you would need to dedicate the allotted time for independent study as well as actively attend the live sessions and lectures.

9. Is there other preparation that I should do before starting the course?

You will be required to use software to prepare written reports and prepare slides to accompany presentations. Our computer facilities (see below) offer you access to Microsoft Excel, PowerPoint and Word for this purpose. If you are not familiar with these programs, you might want to familiarise yourself with them before the course commences. Other software such as R for statistical analysis will be taught on the Course (please see the accompanying R preparation guide). In addition, communication and teaching software such as padlet will be introduced in induction week.

10. Will I need to buy a calculator?

You will need to do calculations for practicals and homework, where you can use laptops and/or mobile phones, so it may not be necessary to buy a calculator.

If you decide to buy a calculator, we recommend that you use the CASIO fx-85GT Plus (£9-10). However, any standard scientific calculator should also be fine, so if you already have one, there is no need to buy another one. Importantly, you should check that the calculator has functions to calculate log (bases 2

and 10), exponentials, arithmetic means, standard deviations (S.D.), variance, geometric means, and square roots. We advise that you become familiar with these basic functions of your calculator before your arrival.

11. What books do I need to buy for the course?

There is a core reading list of texts in addition to the books recommended for your pre-sessional reading. You will receive a handbook containing links to the core reading lists, when you arrive. These key texts will be available in the library (and mostly online), so choosing to buy any of the books will be up to you in case you want to retain a personal copy.

12. Do I need to choose my own summer project topic?

The summer months in Term 3 are dedicated to full-time work on a research project. We provide students with a list of potential summer project topics at the start of Term 2 (in late January or early February), and advise that you start thinking about possible projects before this. Alternatively, you may wish to submit your own project ideas for approval. Your idea for a summer project, must be supported by a member of staff within the School of Public Health who is also willing to supervise the project. If you already have project ideas in mind, please discuss these with your personal tutor or the course organiser soon after your arrival.

Chapter 9

Course Contact Information

If you have any further questions, please consult the following key contacts or resources:

- The Registry team for queries on your application, outstanding conditions, fee status, and registration status.
- Postgraduate Education Co-ordinator, Ms Jo Tite (email: j.tite@imperial.ac.uk), for queries on timetables, induction week or any other administrative issues
- MSc HDA-ML Course Co-Director and Senior Teaching Fellow for queries on the course content, Dr Sabrina Andrade Rodrigues (email: s.andrade-rodrigues@ic.ac.uk)
- Please familiarise yourself with information about postgraduate studies, and further information collated for new postgraduate students before you arrive.

Chapter 10

Mandatory Training

Once you are enrolled as an Imperial student, you will have a series of short mandatory training exercises to complete.

10.1 Data Protection Awareness

The conduct of most epidemiology and public health research requires collection, storage, and processing of personal data. As a controller for most of the personal data it processes, Imperial College London is committed to full compliance with the applicable data protection legislation, including the UK General Data Protection Regulation (GDPR) and the Data Protection Act 2018.

As you may handle personal and sensitive personal data through teaching and research activities, you are required to complete a **Data Protection Awareness** e-learning course, which includes best practice and scenarios for maintaining data security, an overview of the data protection regulations, compliance, rights and enforcement.

You can only complete the course once you are enrolled as an Imperial student, and are encouraged to complete it before the course commences. Once you have enrolled with Imperial, you can access and complete the **Data Protection Awareness** course on Canvas (your learning platform). You will receive more information on this in Induction Week.

10.2 Plagiarism Awareness

As an Imperial College London student, you will be required to use and share academic information ethically, with academic integrity and in accordance with Imperial College's Examination & Assessment: Academic Integrity Policy.

To support compliance with this policy, you are required to complete a Plagiarism Awareness Course, developed by the Graduate School in conjunction with the Library. It aims to equip all Imperial postgraduate students with a working knowledge of the concept of plagiarism and how to avoid it. You should be auto-enrolled to this course; instructions for manual enrolment are also on the College website. More information will also be given in the induction week.

10.3 Attributes and Aspirations (Recommended)

Attributes and Aspirations is an interactive, practical, flexible online short course designed to support you make decisions on your professional future. It was created based on research into the skills employers and PhD programmes look for, so you can be sure they are the skills that you really need. AA offers topics like teamworking, decision making, communication, as well as career planning and interview preparation skills. You can enrol and complete them when you need them on a timeline that works for you! Visit the Skills Map to explore what skills you can practice and where to find them.

By enrolling on AA - You can develop key skills that will help with your Master's study and in your future. - You will access content that designed specifically for Imperial College Postgraduate students in your faculty. - You will receive a certificate for each module you complete, but there are no formal assessments on the course.

Chapter 11

Installing R and R Studio

You must install R before you can install R Studio. A note: R is an open-source project that can be downloaded from the CRAN Repository. R Studio is owned by Posit PBC (a public benefit corporation). It puts out a free version of R Studio and a paid enterprise version. You should not pay for R Studio. You will need to go to two different websites to download R and R Studio.

11.1 Installing R

Please download and install R (version 4.6.1) from the CRAN following the relevant instructions below.

11.1.1 Mac users

R for Mac OS X can be downloaded from the following link: <https://cran.r-project.org/bin/macosx/>

The link will take you to the download page on the CRAN (Figure 1). Please select the latest version by clicking on R-4.6.1.pkg (circled in red) if this version can be supported on your machine. Otherwise, choose the latest supported version. If you have a laptop with the Apple chip, you can select the version circled in green, which has been specifically designed for that chip.

Note: The images below refer to previous versions but the installation steps will be similar for the latest version.

Once the download has finished, you can follow the recommended steps for installation (Figure 2).

As indicated on the download page, you will also need to install XQuartz to be able to use some more advanced R functionalities. This can be done from this link: <https://www.xquartz.org>

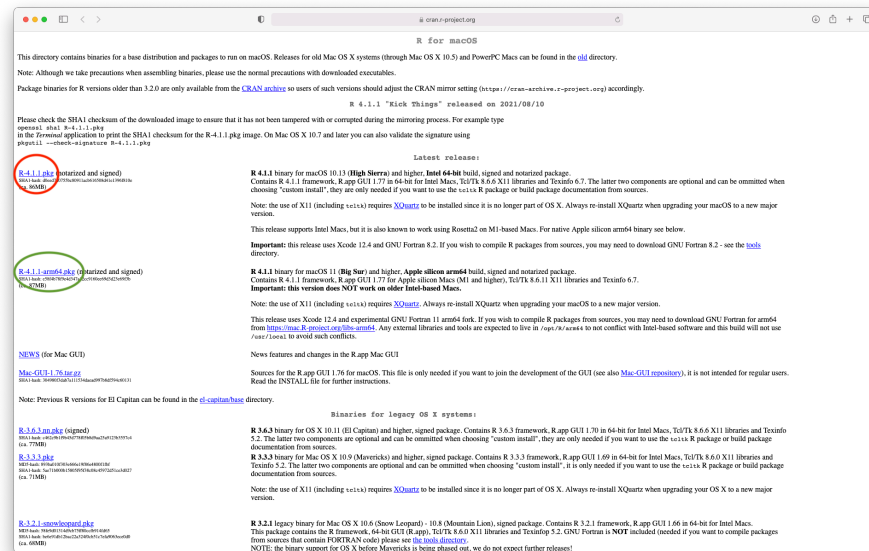


Figure 11.1: Downloading R from the CRAN

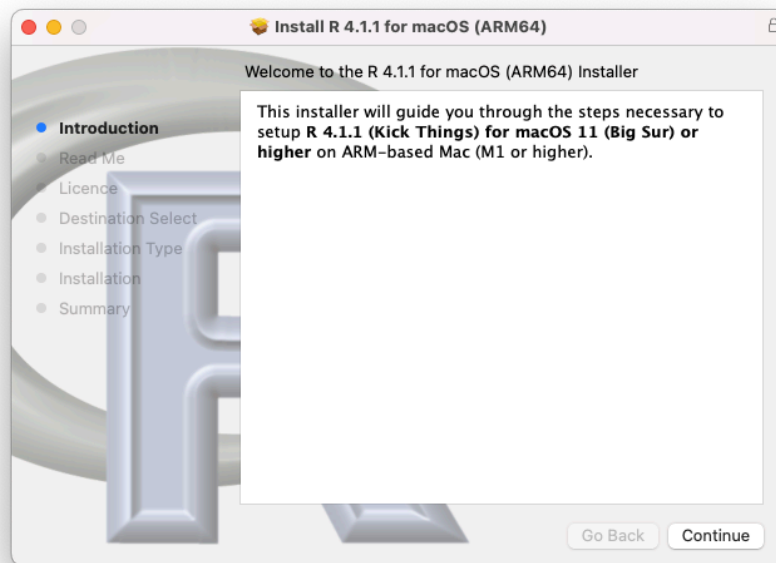


Figure 11.2: R installation

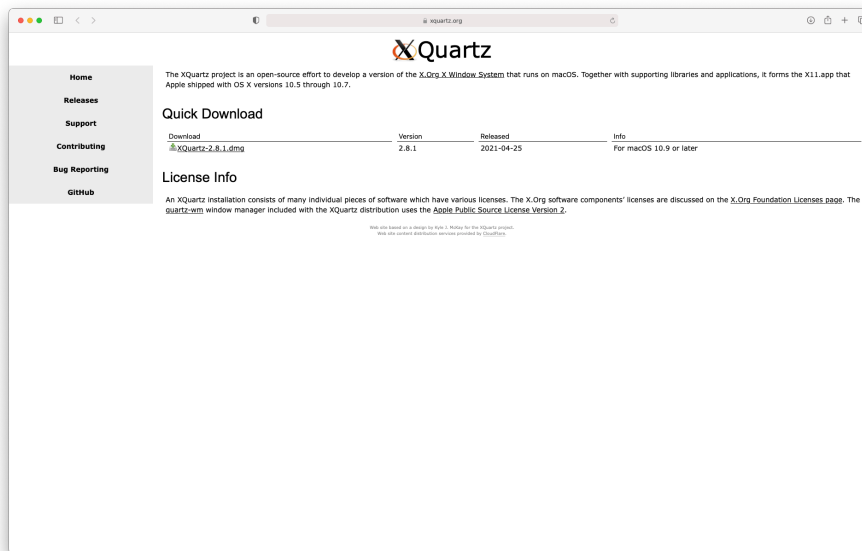


Figure 11.3: Downloading XQuartz

11.1.2 Windows users

R for Windows can be downloaded from the following link: <https://cran.r-project.org/bin/windows/base/>

The link will take you to the download page on the CRAN (Figure 4). Please select the latest version by clicking on “Download R 4.6.1 for Windows”.

11.2 Installing RStudio

R Studio is both a GUI and an integrated development environment for R. You can download the Open Source R Studio Desktop from this link: <https://www.rstudio.com/products/rstudio/download/>

This link will take you to this webpage:

Click on “Download” and select the installation file corresponding to your system. Once the file is downloaded, please follow the instructions for installation.

11.3 Installing Packages

Packages are a collection functions, compiled code, and sample data. At their core, they are just useful extensions to programming languages that you can download from the internet. Browser plug-ins/extensions for a web-browser

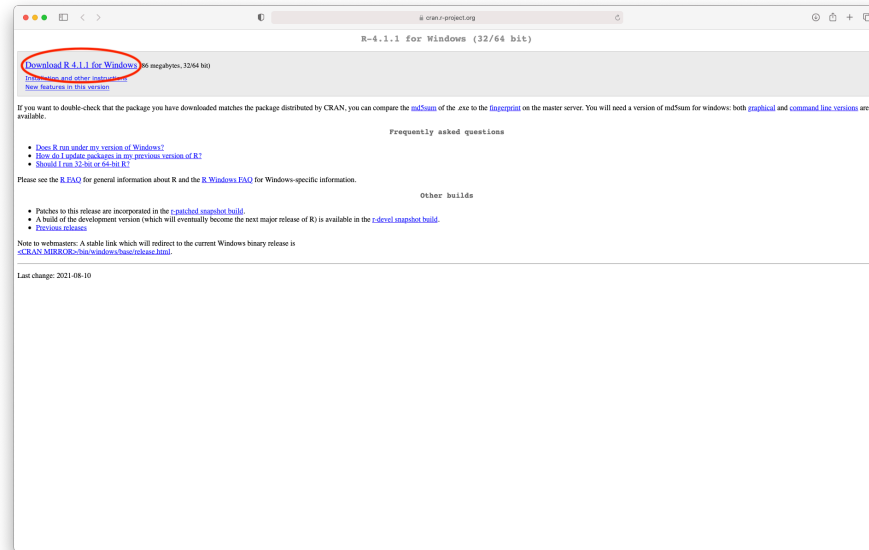


Figure 11.4: Downloading R from the CRAN

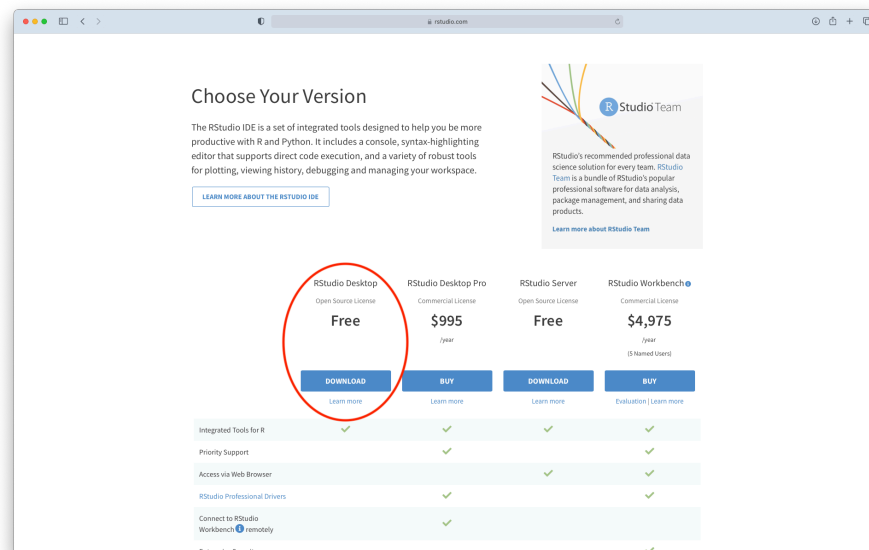


Figure 11.5: Downloading R Studio (step 1)

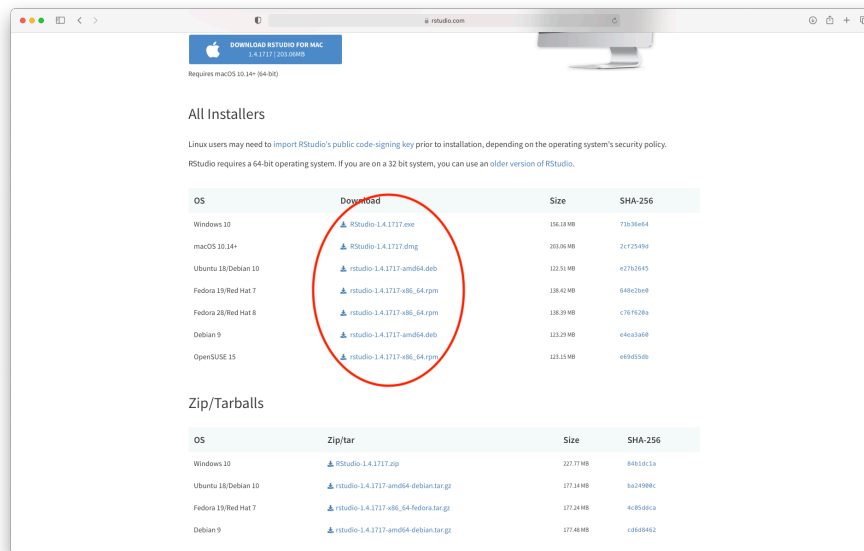


Figure 11.6: Downloading R Studio (step 2)

such as Google chrome are essentially “packages” that allow you to add a useful function that Chrome doesn’t current have.

Open-Source software, such as R, is software for which the original source code is made freely available and may be redistributed and modified. Users are able to write code to perform common tasks and share it freely by putting their work on a repository. A software repository, or repo, is a centralized storage location for software packages, code, and related files. Think of it as a digital library where developers can store, access, and manage different versions of their work.

There are two main repositories of R packages (sets of built-in R functions): the CRAN (Comprehensive R Archive Network) and Bioconductor. R checks only CRAN by default, by you can tell it to look elsewhere (such as Bioconductor). The code below will install most of the R packages that you need in the course. You don’t need to understand all of the code, but can you spot where it is telling R to check Bioconductor instead of CRAN?

Once both R and R Studio are installed, you can open RStudio and install the required packages by running the following code (you can copy it by clicking the pages icon in the top right of the code block below). We recommend opening R Studio, going to File -> New File -> R Script, copying the code below, pasting it into the R Script you created, and running the entire script at once.

```
### First we create a function that checks if you have the package installed
### and, if you don't have it, installs the package
```

```

checkInstallPackage <- function(package.list){
  new.packages <- package.list[!(package.list %in% installed.packages()[,"Package"])
  print(paste(length(new.packages), "packages require installation. Installing now"))
  if(length(new.packages)) install.packages(new.packages)
}

### Now we have a function, we can pass a list of the packages we will be
### using on the course into the function, and install them all in one go
# first we create the list of packages
package.list <- c("e1071", "optparse", "tidyverse", "mvoutlier", "pcaMethods",
  "imputeLCMD", "lme4", "RColorBrewer", "VennDiagram", "glmnet", "omics",
  "utils", "dplyr", "ROCR", "ggplot2", "ggfortify", "survival", "igraph",
  "ppcor", "abind", "parallel")

# now we run the function
checkInstallPackage(package.list)
# this may take some time to run #
### Installing some packages from Bioconductor ###
# some of the packages we use are only available on bioconductor and these need
# to be installed separately. Run this code to install these packages.
# You may receive a prompt "Update all/some/none? [a/s/n]:" -
# if you see this, type "a" and press enter
if (!"pcaMethods" %in% rownames(installed.packages())) {
  if (!requireNamespace("BiocManager",
    quietly = TRUE))
    install.packages("BiocManager")
  BiocManager::install("pcaMethods")
}
if (!"impute" %in% rownames(installed.packages())) {
  if (!requireNamespace("BiocManager",
    quietly = TRUE))
    install.packages("BiocManager")
  BiocManager::install("impute")
}

```

Once you have run all these lines of code, please make sure that all the packages have been properly installed. They should all be listed in the “Packages” tab (bottom right hand quadrant of R Studio):

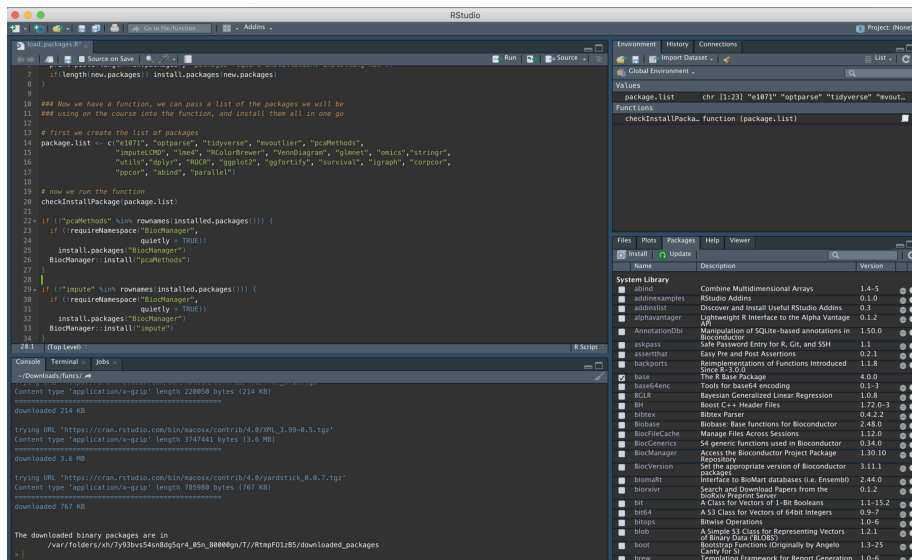


Figure 11.7: Checking package download

Chapter 12

Installing Python, Anaconda, and VSCode

Python is a popular language for research computing, and great for general-purpose programming as well. It requires installing packages for most analytical tasks. While it is possible to use an all-in-one installer to download Python and a large portion of the packages that most researchers need, HDA will have you install the packages as you go to gain experience with programmatically downloading packages.

Users employ a wide range of IDEs to work in Python. The three most common are Jupyter Notebooks, Pycharm, and VSCode. Anaconda is a Python distribution which has created a popular package and environment manager called conda. In simple terms, if you download Anaconda, you get a bundle of popular programs and IDEs that you can use to work in Python. Conda is essentially Python's equivalent of CRAN for R – where you can access packages and code written by other users.

Note: This is an over simplification of what conda is. It allows users to build environments, download packages containing any code, and can handle complex dependencies. You will be introduced to these concepts further in the course and how to use conda. There is another “repository” for Python called pip that only installs python packages that is more equivalent to CRAN. There are also conventional repositories such as PyPI.

For now you just need to know that you need to install Python, Anaconda, and the VS Code.

12.1 Python

The latest stable version of Python is 3.13.3. Please download the “installer” which is suitable for your computer specifications. If you are unsure what option to select, please see the Computer Specifications section of this guide.

Python for Mac

Python for Windows

12.2 Python IDEs

There is no best IDE. HDA will primarily use Jupyter Notebooks and VS Code to familiarize you with industry standard tools. Additionally, being comfortable into VS Code will make interacting with advanced computing environments that don’t have as user friendly of GUIs easier - such as Imperial’s High Performance Computing Server. You may choose a preferred option for any independent work (not for practical sessions) and may be forced to use a different IDE at your next step.

Why is there not one best IDE? Each one is designed with a different purpose in mind. Some are meant to use sharing. Others to appeal to scientist. There are also a large number that focus solely on software development. As always, you must pick the right tool for the right job.

12.2.1 Anaconda

Please download the “installer” which is suitable for your computer specifications. If you are unsure what option to select, please see the Computer Specifications section of this guide.

Download Anaconda

You do not need to sign up for an account even if prompted.

After clicking on the link above, you will need to select the “Skip registration” option on the resulting website (the red circle in the picture below):

Please select the “Distribution Installers” option (red arrow in the picture below):

Note: The difference between the Distribution and Miniconda Installers is what is included in the download package. The distribution installation comes with a variety of useful packages and programs that you can easily install. If you install the Miniconda then you will have to manually install all of the following IDEs (which is not recommended).

Distribution

Register to get everything you need to get started on your workstation including Cloud Notebooks, Navigator, AI Assistant, Learning and more.

- Easily search and install thousands of data science, machine learning, and AI packages
- Manage packages and environments from a desktop application or work from the command line
- Deploy across hardware and software platforms
- Distribution installation on Windows, MacOS, or Linux

Free Download

Get access in 30 seconds. Completely free.*

[Get Started >](#)[Returning Users >](#)

*Subject to our [Terms of Service](#). Use of Anaconda's offerings at an organization of more than 200 employees/contractors requires a paid business license unless your organization is eligible for discounted or free use. [See Pricing](#).

Skip registration

Figure 12.1: First Page of Anaconda Website

Home / Free Download

Download Now

Download Anaconda Distribution or Miniconda by choosing the proper installer for your machine. Learn the difference from our [Documentation](#).

Distribution Installers

Download for Mac

▼

For installation assistance, refer to [troubleshooting](#).

Miniconda Installers

Download for Mac

▼

For installation assistance, refer to [troubleshooting](#).

Figure 12.2: Select Distribution Installers

12.2.2 Jupyter Notebook, Jupyter Lab, Pycharm, and Spyder

Once Anaconda has been installed, please open the application (this can take a little while). Once it is open, it should look like this:

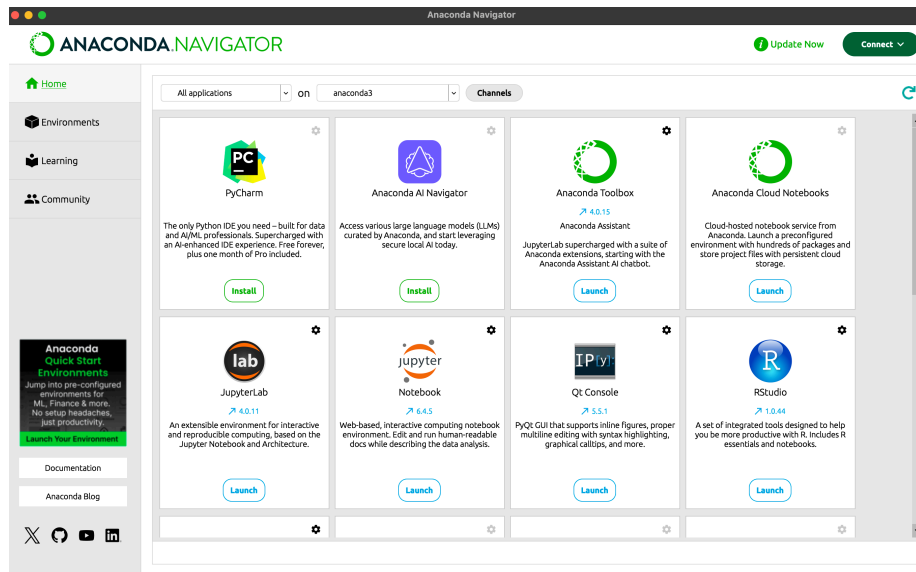


Figure 12.3: Anaconda Program Manager

Note that there are two options: “Install” for programs you do not have and “Launch” for those that are already installed. Please click “Install” for PyCharm, JupyterLab, Jupyter Notebook, and Spyder (you may need to scroll down to find them all). We will not use the other options in HDA as they are not useful or commonly used.

The Jupyter Notebook is the original web application for creating and sharing computational documents. It is commonly used for informal reports due to its ability to handle text and code in a neatly formatted manner. Jupyter Lab adds a file structure ribbon to the Jupyter Notebook layout to easily browse your computer for files to open. They are essentially equivalent and using one over the other is down to personal choice.

Spyder is very similar to R Studio with a panel lay out with a plotting window and console to visualize the output of code:

Pycharm is a big deviation from the GUIs that you have seen so far. It is designed for software development so does not have a built-in window to visualize the results of the code (such as the plots). This can be disorientating for a scientist looking to write analytical code. However, the large suite of additional

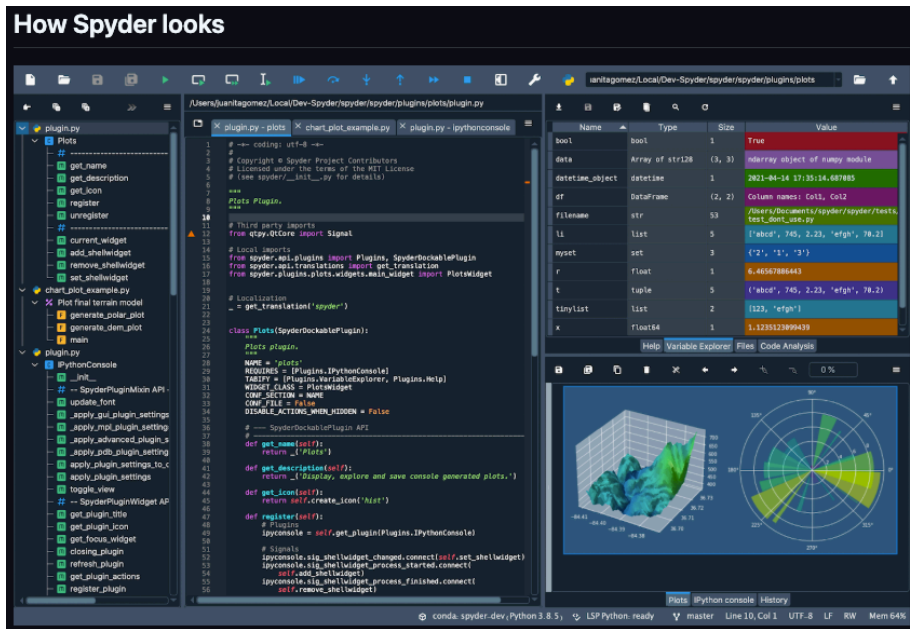


Figure 12.4: What Spyder Looks Like

functions make Pycharm a powerful development environment for complex coding projects.

12.2.3 VS Code

Visual Studio Code, commonly referred to as VS Code, is an integrated development environment developed by Microsoft for Windows, Linux, macOS and web browsers. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded version control with Git.

“Visual Studio Code remains the preferred IDE across all developers, increasing its use among all developers this year from 75 percent to 81 percent.” - the Stack Overflow 2023 Developer Survey

Download VS Code

****Please only do the first step of the instructions (red rectangle in the picture below)**.** You will go over the other options during the course when necessary. We do not recommend attempting to modify the base install for now as it can be very overwhelming and easy to install things incorrectly.

What does VS Code look like?

It looks like Pycharm! Both are development focused IDEs that do not have a

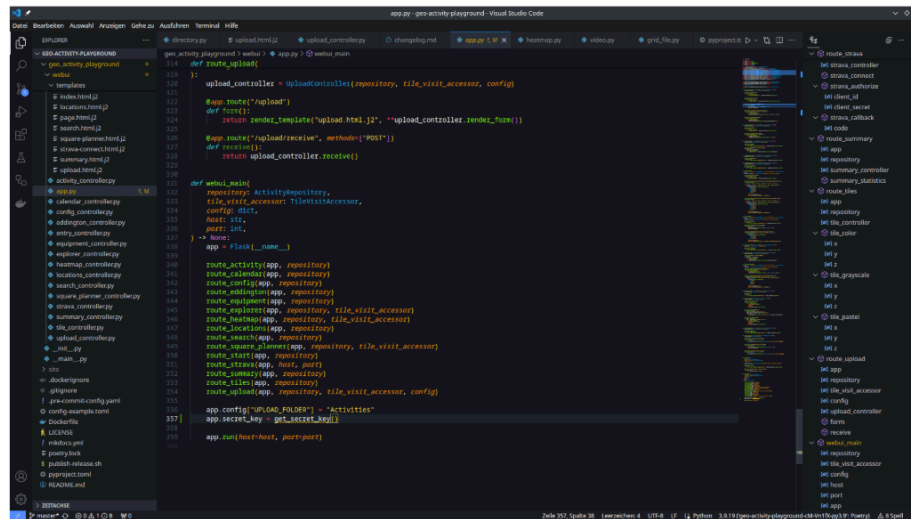


Figure 12.5: What Pycharm Looks Like

graphical display for plots. So why is VS Code so much more widely used than Pycharm?

12.2.4 Pycharm vs. VS Code

Pycharm is better for pure Python development as it has more features, refactorings, powerful fuzzy search and slightly more helpful code completion and tooltips.

VS Code is better for projects with multiple programming languages and it has less advanced but more broad support. With VS Code you can switch between Python, Python Notebooks, R and Bash with ease.

12.3 Package Installation

Users should install Python packages in a programmatic manner which will be taught during the course term. Students will be guided through package installations needed for each practical session.

Set up VS Code for your platform

- 1 Download and install Visual Studio Code for your platform
 - [macOS](#)
 - [Linux](#)
 - [Windows](#)

Note

VS Code ships monthly releases and supports [auto-update](#) when a new release is available.
- 2 [Install additional components](#)

Install Git, Node.js, TypeScript, language runtimes, and more.
- 3 [Install VS Code extensions from the Visual Studio Marketplace](#)

Customize VS Code with themes, formatters, language extensions and debuggers for your favorite languages, and more.
- 4 [Enable AI features](#)

Tip

If you don't yet have a Copilot subscription, you can use Copilot for free by signing up for the [Copilot Free plan](#) and get a monthly limit of completions and chat interactions.
- 5 [Get started with the VS Code tutorial](#)

Discover the user interface and key features of VS Code.

Figure 12.6: Install VS Code

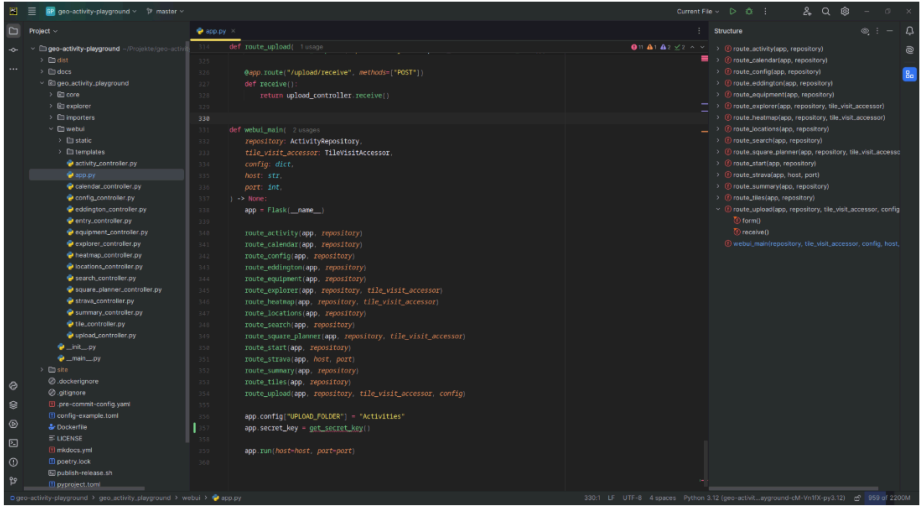


Figure 12.7: What VS Code Looks Like

Chapter 13

Installing MySQL and DBever

You will be asked to create a password for mySQL. You will need it every time you want to use mySQL. Please write it down somewhere and do not lose it! Note: This is terrible information security practice. However, you mySQL password for an instructional course is not something that you need to protect. We ask the students to do this each year, but a portion of them always forget.

SQL (Structured Query Language) is a standard language used to store, retrieve, and manipulate data in relational databases. It allows end-users to communicate with databases and perform tasks like creating, updating, and deleting databases. It underpins most modern technonology. While there are a host of careers that revolve entirely around SQL, any analyst is expected to know enough to be able to retrieve the information that they need. While some organizations use Excel for their data management, this is not scalable and leads to unforeseen errors.

Excel Loses 16,000 Covid Cases

13.1 MySQL Installation

The latest stable release is MySQL 8.0.36. While there are newer versions available, they still contain bugs that have yet be to patched (or found in some cases).

Follow the instructions depending on the type of operating system you are running.

Note: To re-emphasize, you will be asked to create a password along the process. Please write it down somewhere!

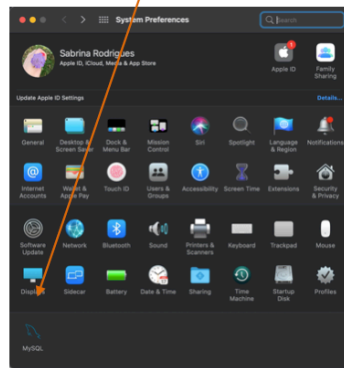
13.1.1 Mac users

If you are using a Mac OS of version < 10.14 and MySQL 8.0 does not work, you can install an older version of MySQL by going to this [Archive](#) and download MySQL 5.7.x that suits your Mac OS version.

Follow These Instruction Up to Step 8

Once your installation is done you can test if it was successful by doing the following:

1. Open up System Preferences and double click on



2. You should see something like this:

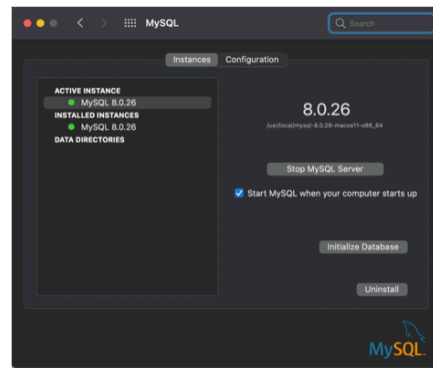


Figure 13.1: Mac MySQL Check

13.1.2 Windows users

If you are using a Windows of version < 10, you can install an older version of MySQL by going to this [Archive](#) and download MySQL 5.7.x that suits your Windows version.

For those with Windows 10 or higher, please install MySQL 8.0:

Windows Install

Once your installation is done you can test if it was successful by doing the following:

13.2 DBeaver Installation

DBeaver is a free, universal database tool and SQL client. It's designed for developers, database administrators, and analysts who work with various databases, including both SQL and NoSQL types. DBeaver offers a user-friendly interface for managing, developing, and analyzing databases, with features like code com-

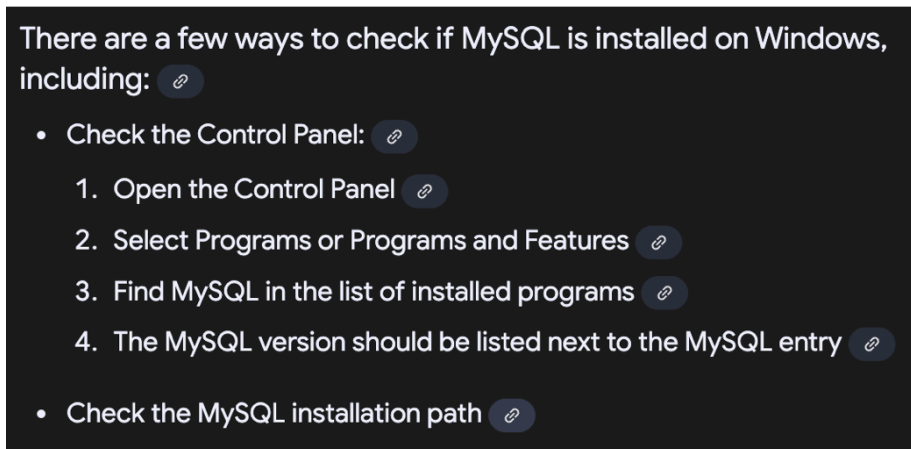


Figure 13.2: Windows MySQL Check

pletion, syntax highlighting, and a plugin architecture for customization. There are paid options, but you do not need to purchase anything.

There are other SQL GUIs or IDEs, but there is no consensus industry favourite. Unlike the other programming languages you have seen so far, SQL has closed-source versions (not free). Both Microsoft and Oracle have their own version of SQL (which is very similar to MySQL) which their applications run on. Unlike R or Python, if you are working on a commercial product that has a license from either of those companies, you can call up their support center to get help with your issues or contract out large scale development tasks to their companies.

Please download the installer for DBeaver Community 24.1.4 that matches your operating system:

DBeaver Install

Mac Users will need to drag the downloaded program into the Applications folder:

13.2.1 Testing out DBeaver

This exercise will be covered in the first week of the Program (along with other material). Don't worry if you can't complete it or if something doesn't work. You will have a live session with support to troubleshoot any problems.

When you open DBeaver for the first time, you will be asked if you want to create a sample database. Say yes.

If you said no accidentally, follow the instructions at this link to create the sample database <https://dbeaver.com/docs/dbeaver/Sample-Database/>

Download

DBeaiver Community 24.2.1 Released on September 22nd 2024 (Milestones). It is free and open source (license). Also you can get it from the GitHub mirror. System requirements.	DBeaiver PRO 24.2 Released on September 9th, 2024 PRO version website: dbeaver.com Trial version is available.
 Windows • Windows (installer) • Windows (zip) • Chocolatey (<code>choco install dbeaver</code>) • Install from Microsoft Store  Mac OS X • MacOS for Apple Silicon (dmg) • MacOS for Intel (dmg) • Brew Cask (<code>brew install --cask dbeaver-community</code>)  Linux • Linux Debian package (installer) • Linux RPM package (installer) • Linux (tar.gz) • Linux (tar.gz without Java included) • Linux ARM (tar.gz without Java included)	DBeaiver PRO features: • Advanced security: master password and strong credentials encryption to secure and easy database connection. • Enterprise-level authentication methods: SAML, SSO, OKTA, and Kerberos. SSO authentication cloud services such as GCP, AWS, and Azure. • Advanced SQL databases support: MySQL, PostgreSQL, SQL Server, Oracle and others • NoSQL databases support: MongoDB, Cassandra, Redis, CouchDB and others • Cloud databases support: Redshift, Google BigQuery, Oracle Cloud and others • Native cloud support for Google Cloud, AWS, and Azure. • Access all files in any region with the S3 browser to upload, store, share, and save files like in a regular file system. • AI assistant in SQL and Visual Query Builder to create complex SQL scripts automatically • Database schema development with ERD Edit Mode. • Mock data generation with thousands of entities of different types for database testing. • Comparison of data and schemas between sources to navigate through all the differences. • Query Execution plan graph mode to view and estimate the

Figure 13.3: DBeaver Free vs Paid

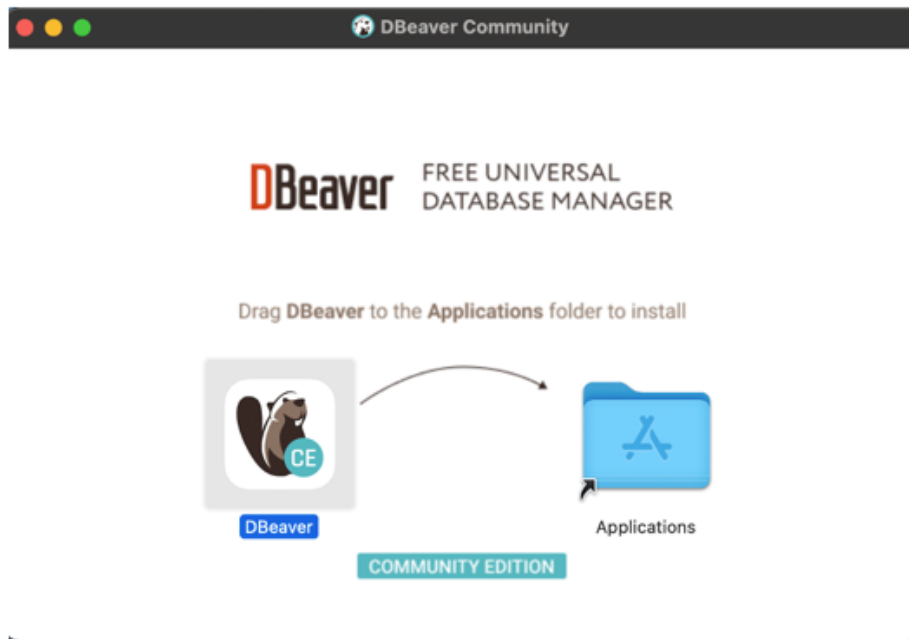


Figure 13.4: Installation DBeaver on Mac

Your screen should look like this:

Now, click the '>' next to "DBeaver Sample Database (SQLite)" (where the red arrow is pointing in the picture above).

You will be asked to Download the SQLite driver file:

Before you start testing DBeaver using a local SQLite server. Let's go over some definitions:

Local = On your computer

Server = Hardware where data is stored (can be local, remote, cloud, etc.)

SQLite is the dialect of SQL that the database is set up to be queried in.

In DBeaver, create a New SQL script and save it with an informative name:

What does the writing above your script (the red arrow above) mean?

Write the two commands in the picture below into your script.

After writing each line (with your cursor at the end of the line), press the orange arrow (1 in the picture).

What happened?

When you have entered both press the orange arrow inside the scroll (2 in the picture).

What happened?

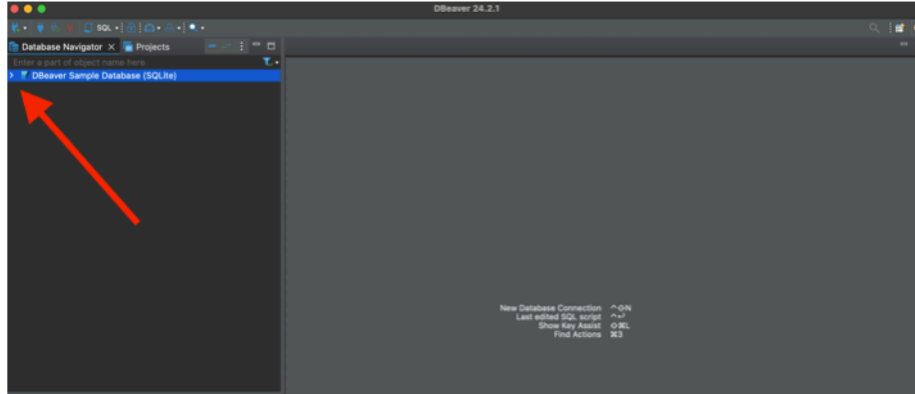


Figure 13.5: What DBeaver Looks Like

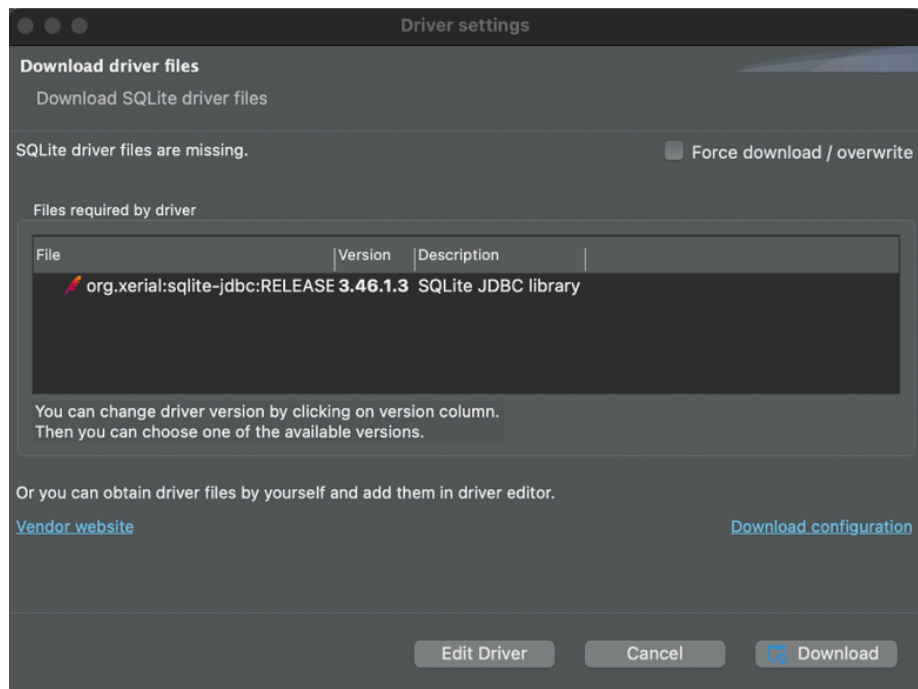


Figure 13.6: Downloading SQLite Driver File

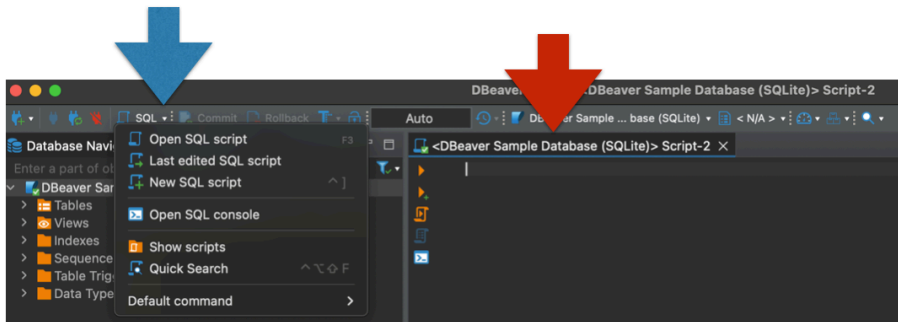


Figure 13.7: DBeaver New Script

Switch the order of your lines of code and press (2) again. Is your output any different?

Save your script

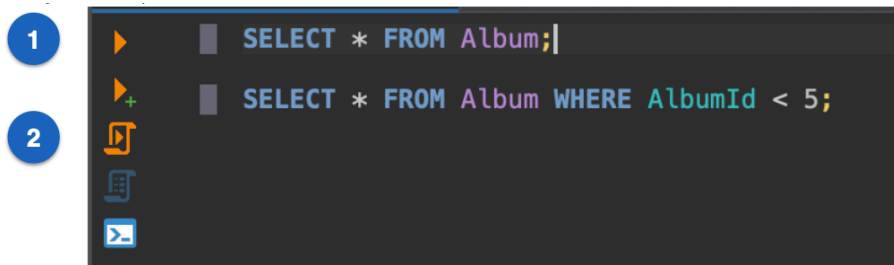


Figure 13.8: Running SQL Commands

You have just queried an instance of a local server using a SQLite relational database management system in an Integrated Development Environment. Don't get overwhelmed by any confusing or complex sounding jargon though – when you start at the beginning and keep practicing you will be writing complex code before you know it.

A last note: HDA teaches MySQL, but you just used SQLite. Does this mean that the commands you ran won't work in MySQL? No. Remember that most dialects of SQL have a vast overlap. These commands will work in SQLite, MySQL, SQL Server, and more. You don't need to worry about the other dialects for now, but remember that being an expert in one version of SQL will make it trivial to become an expert in another.

Chapter 14

Installing Git, Github, and Github Desktop

Collaboration is at the heart of programming and research. Git allows a group of programmers or researchers to work together on a project in an organized and systematic manner. You may have worked on a group project where the files being shared have increasingly ridiculous names – `project_v3`, `project_v3_final`, `project_v3_final_final`, etc. You may have also updated a version of a project, only to find out that you had been working on an older version. Git and Github solves all of these problems, and more.

14.1 Git

Git is a version control system that lets you track who made changes to what when and has options for easily updating a shared or public version of your code on github.com. Version control is a system that tracks and manages changes to files over time, primarily used in software development to manage changes to source code. It allows teams to collaborate, track modifications, compare different versions, and revert to previous states, ensuring a history of changes and minimizing potential disruptions. For example, if you change your project code and break everything, you are able to instantly change your code back to earlier versions (that still worked).

It is the most widely used language in programming with every project that is being written in every language using Git (with minor exceptions). Note: There are other version control systems such as Mercurial and Fossil but they are niche applications with far less utility than Git.

The latest version is 2.55.0. You may already have Git on your computer (it is included in the install of various other programs).

To check, please open the Terminal app, type `git --version` and press Enter/Return. If you do not have Git, you will be provided instructions to follow. Alternatively, you may close the Terminal and go to the git website to download an installer:

Install Git

Please select the installer download for your system and computer specifications. If you are unsure which installer to select, please see the Computer Specifications section of this guide.

14.1.1 Note for Mac Users

If following the instructions provided by Git in your Terminal, follow the instructions to Install the “command line developer tools”. Do not click “Get Xcode”, because that will take too long and is not necessary for our Git needs. If you require at a later date, it will be easy to install then. After installing these tools, there won’t be anything in your /Applications folder, as they and Git are command line programs.

For older versions of OS X (10.5-10.8) use the most recent available installer labelled “snow-leopard” (look for the file name with snow-leopard in it): **Install Git for OS X 10.5-10.8**

Note: this project is no longer maintained. Because this installer is not signed by the developer, you may have to right click (control click) on the .pkg file, click Open, and click Open in the pop-up dialog.

14.1.2 Updating Git

If you have git, but an older version than 2.55.0, then you can update your git version - using git. Type the following into your Terminal and press Enter:

```
git clone https://github.com/git/git
```

14.2 Github

You will need a Github account for this program and for your next steps. Employers often ask to see your github as part of your application process. Every company writing software of any type uses Github. Like everything else in this guide, you don’t need to be an expert with Github now - but the more you use these unfamiliar tools the easier it becomes.

You will need an account at github.com for parts of the Git lesson. Basic GitHub accounts are free (you don’t need more than a basic account).

Create a Github Account

There is nothing else you need to do with the account for now, but please make sure that the password is saved to make it easy to access in the future.

14.3 Github Desktop

Github Desktop is one of the many GUIs for Github. Like other GUIs, it allows you to interact with Git and Github without using the command line and has useful visualizations to make what you are doing much easier.

“GitHub Desktop is really one of the best 80/20 programs I have seen. It hides the details of git (which are irrelevant) and just lets you get on with your work” - from the promotional material of Github Desktop.

Install Github Desktop

When you open Github Desktop, it should look like this:

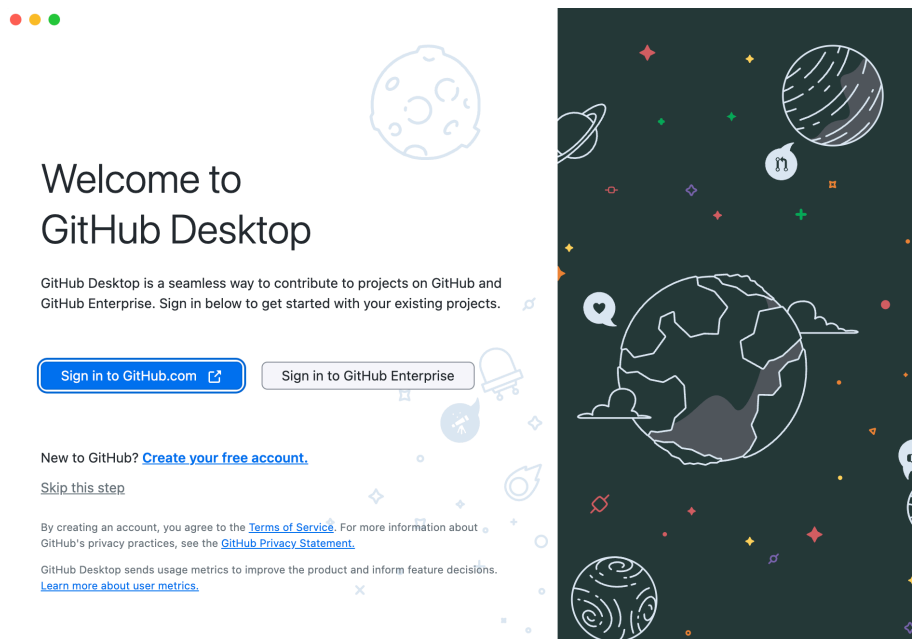


Figure 14.1: Opening Github Desktop

Click sign in to Github.com and follow the instructions. Once you are finished, your Github Desktop should now look like:

That is all for now. If you want to start becoming familiar with Github, click on the “Create a Tutorial Repository” in Github Desktop to be walked through the basics.

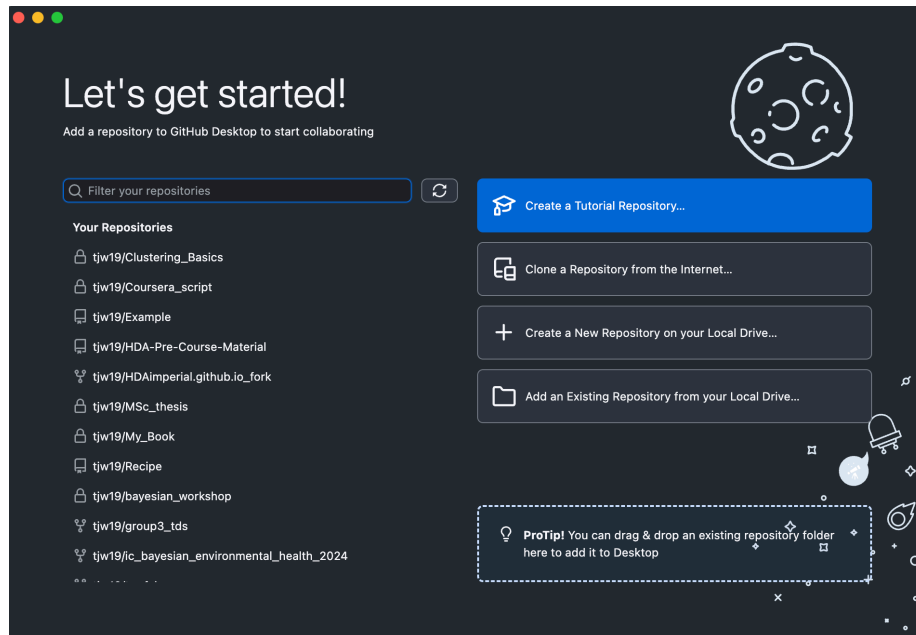


Figure 14.2: Github Desktop

Did you notice the word “repository” cropping up again? You had previously been told that CRAN was a repository for R where you could download approved R packages. You are now making your own repositories. While they are not official (like CRAN), other users can download your code and use it if you do not set your repository to private (which is not a bad thing!).

Chapter 15

Change Log

The current version of this Guide is v.0.2.

v.0.0

The first version of the guide contains all of the key information on HDA and how to prepare for the program.

v.0.1

Minor updates to correct minor typos after review by the administrative team of all term dates and contact information.

v.0.2

Previous versions were created by Mr Thomas Wright, a previous teaching fellow and student of the MSc HDA-ML. The current version has been updated by Dr Sabrina Rodrigues and updates include:

- corrections to minor typos
- changes to dates
- corrections of figure numbering
- updates to latest release numbers

If you notice any typos or some releases need to be updated, please contact Dr Sabrina Rodrigues with any changes or suggestions.